

04-A

栈与队列

栈接口与实现

君子求諸己，小人求諸人

陛下用群臣，如積薪耳，後來者居上

明主之官物也，任其所长，不任其所短

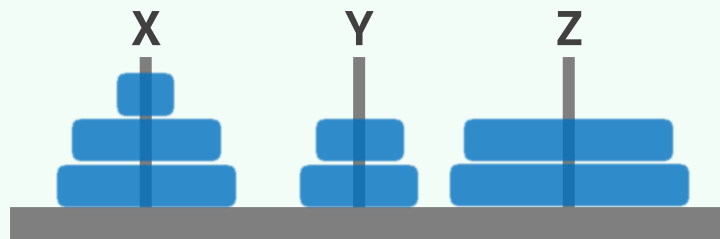
邓俊辉

deng@tsinghua.edu.cn

操作与接口

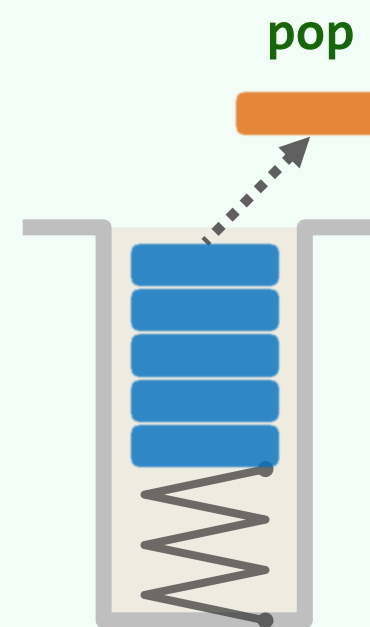
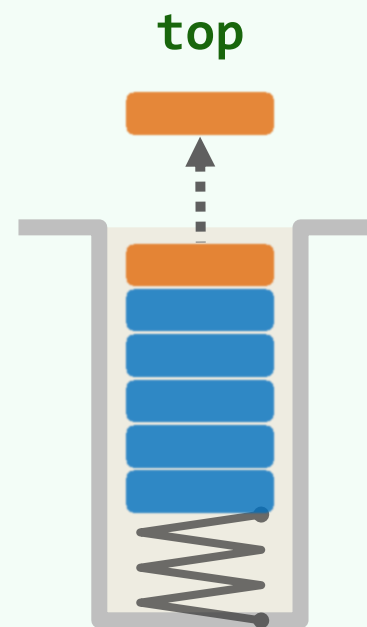
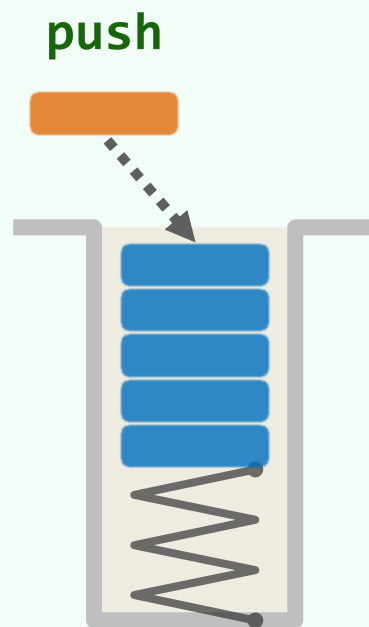
❖ 栈 (stack) 是受限的序列

- 只能在栈顶 (top) 插入和删除
- 栈底 (bottom) 为盲端



❖ 基本接口

- `size()` / `empty()`
- `push()` 入栈
- `pop()` 出栈
- `top()` 查顶



❖ 后进先出 (LIFO)

先进后出 (FILO)

实例

操作	[-- 栈 --x				输出
Stack()					
empty()					true
push(E)	E				
push(C)	E	C			
pop()	E				C
push(G)	E	G			
push(C)	E	G	C		
top()	E	G	C		C
empty()	E	G	C		false

操作	[-- 栈 --x						输出
push(K)	E	G	C	K			
size()	E	G	C	K			4
push(F)	E	G	C	K	F		
empty()	E	G	C	K	F		false
push(G)	E	G	C	K	F	G	
pop()	E	G	C	K	F		G
pop()	E	G	C	K			F
top()	E	G	C	K			K
size()	E	G	C	K			4

实现：既然属于**线性序列**，故可直接基于向量或列表**派生**

```
template <typename T> struct Stack: public Vector<T> {  
    void push( const T& e ) { insert( e ); } //入栈  
    T pop() { return remove( _size - 1 ); } //出栈  
    T& top() { return _elem[ _size - 1 ]; } //取顶  
};
```

❖ 如此实现的栈接口，均只需 $\mathcal{O}(1)$ 时间

❖ 颠倒过来，以向量**首/末端**为栈**顶/底**呢？

❖ 基于List，也应该可以派生出Stack——各接口的效率如何？有何讲究？

04-B1

栈与队列

调用栈：原理与空间

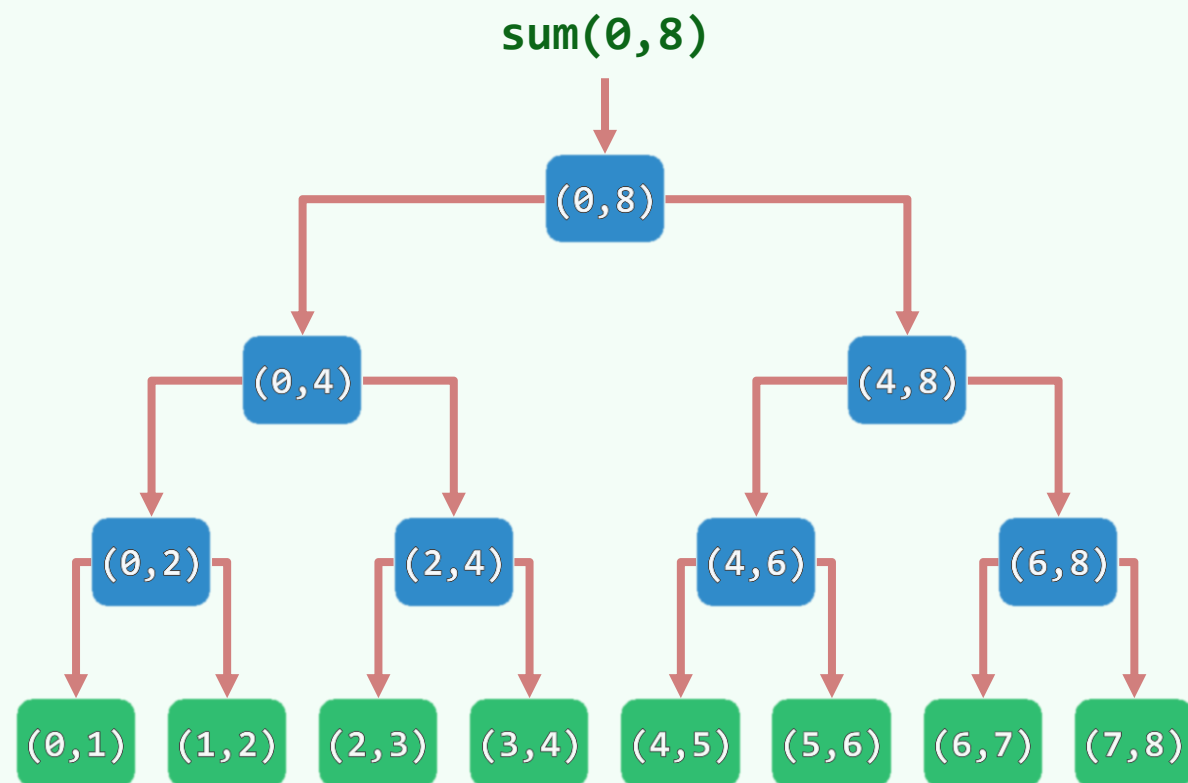
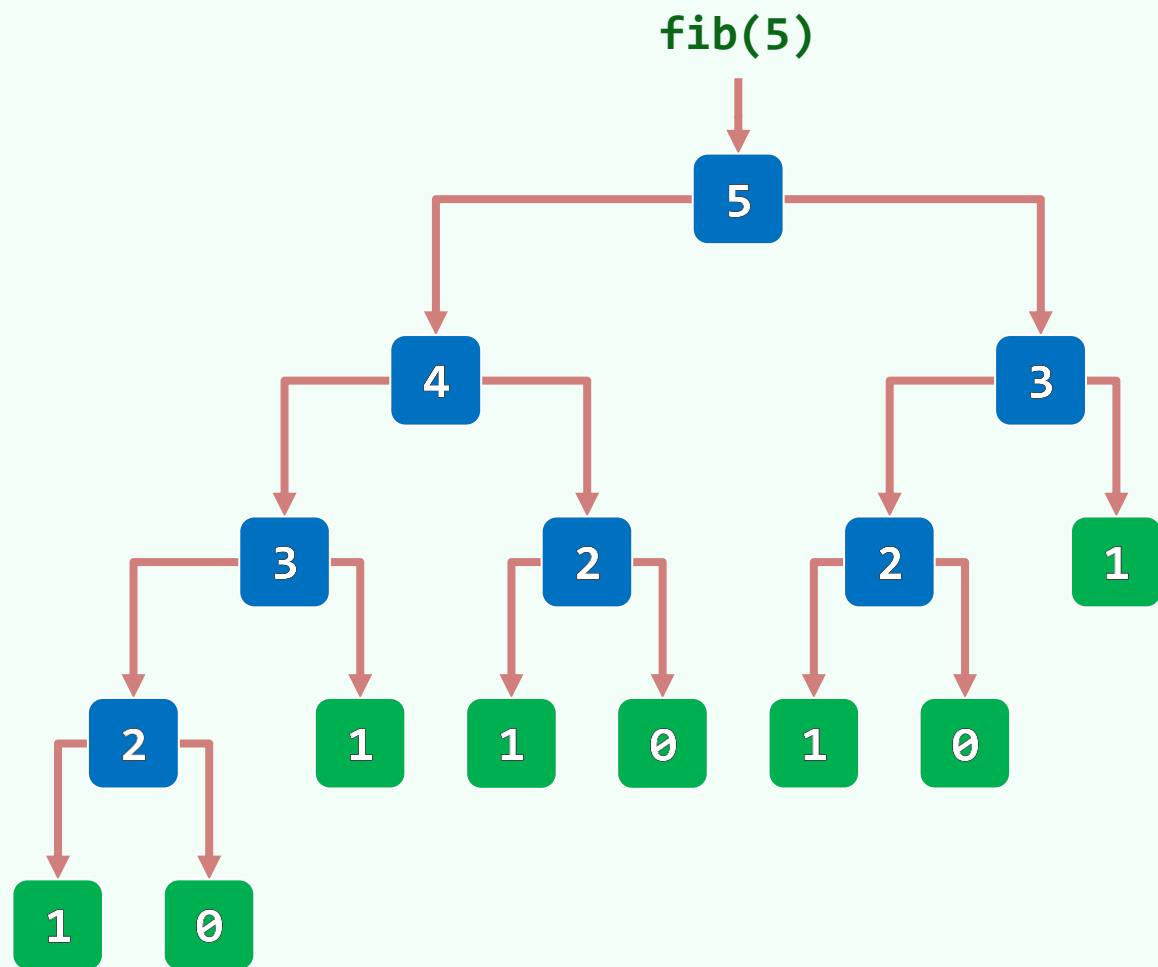
Yessiree. We do not doubt his word, an stack
ourselves into the bus like flapjacks.

命运把我们的大脑当作一个容器，不停地把各种见解装进去、取出来，但
总是现在的和最后的那个见解是可靠没错的

邓俊辉

deng@tsinghua.edu.cn

函数调用树：如何实现？ Theseus的线团 + 粉笔



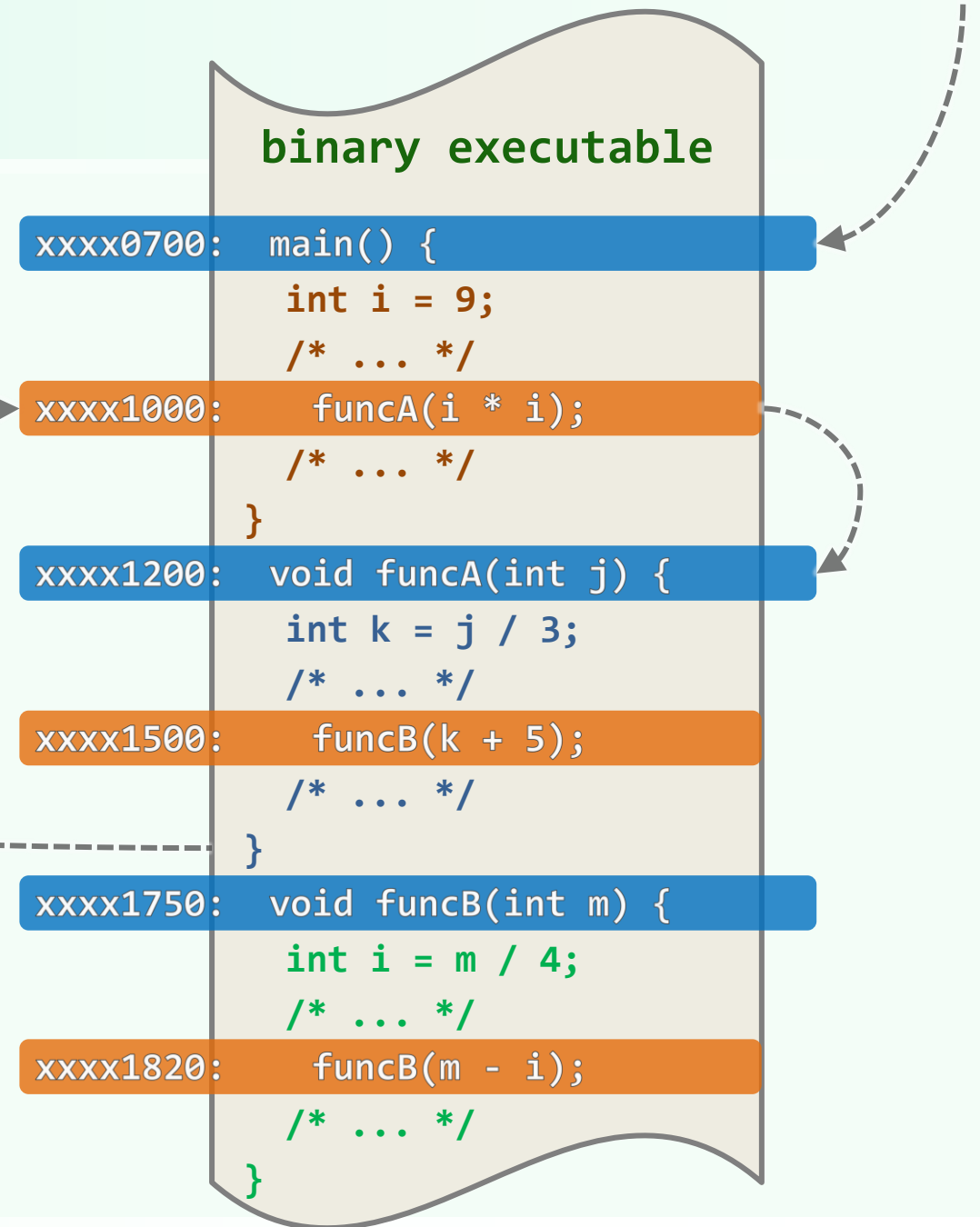
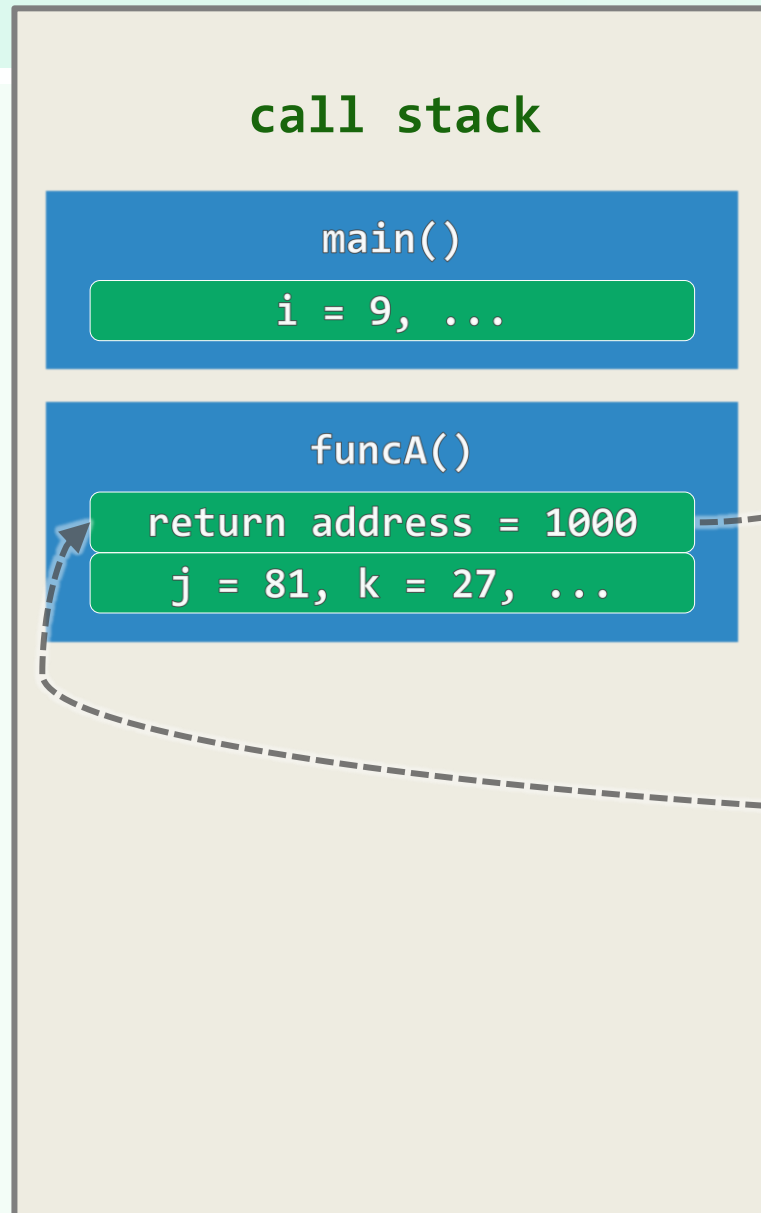
call stack

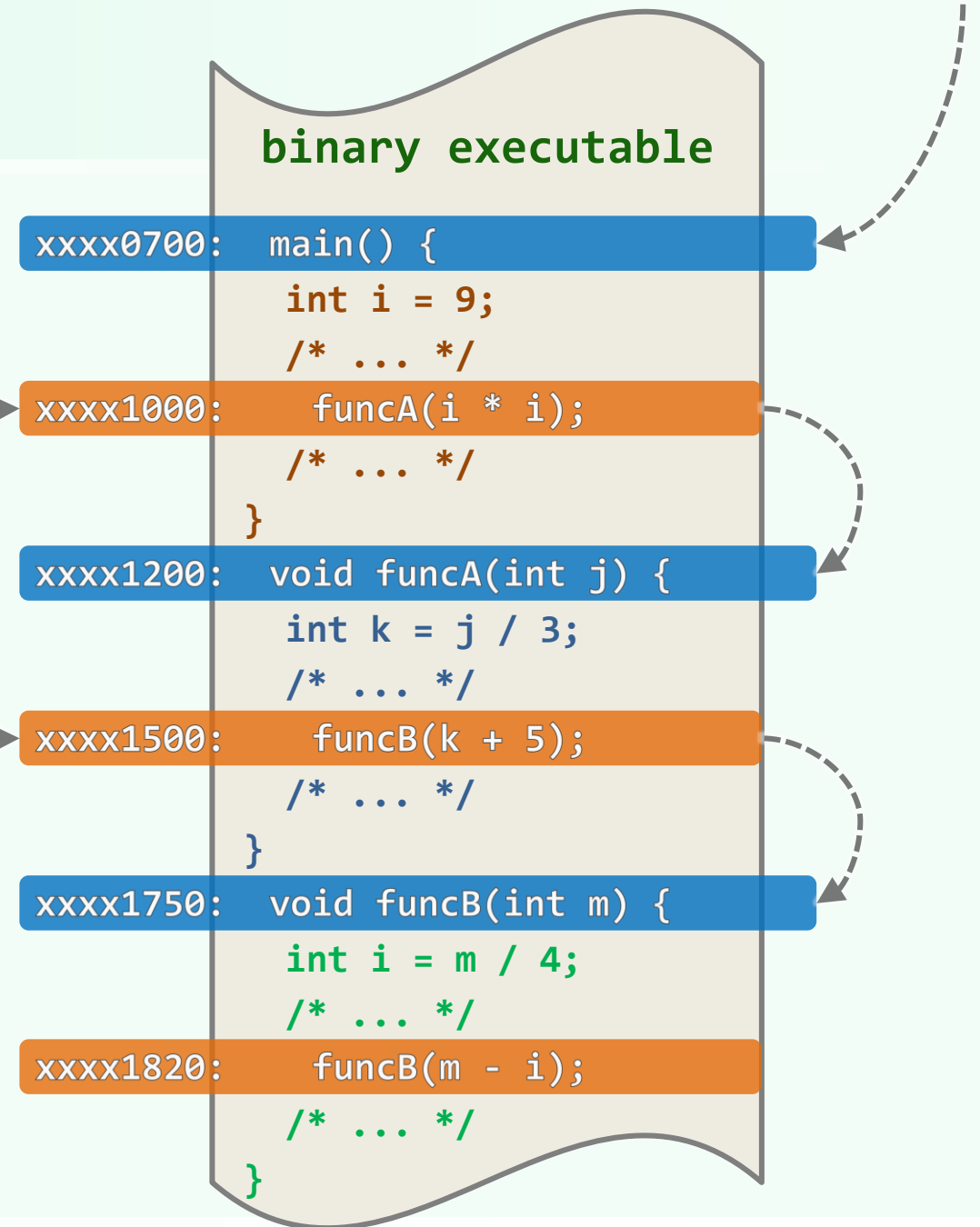
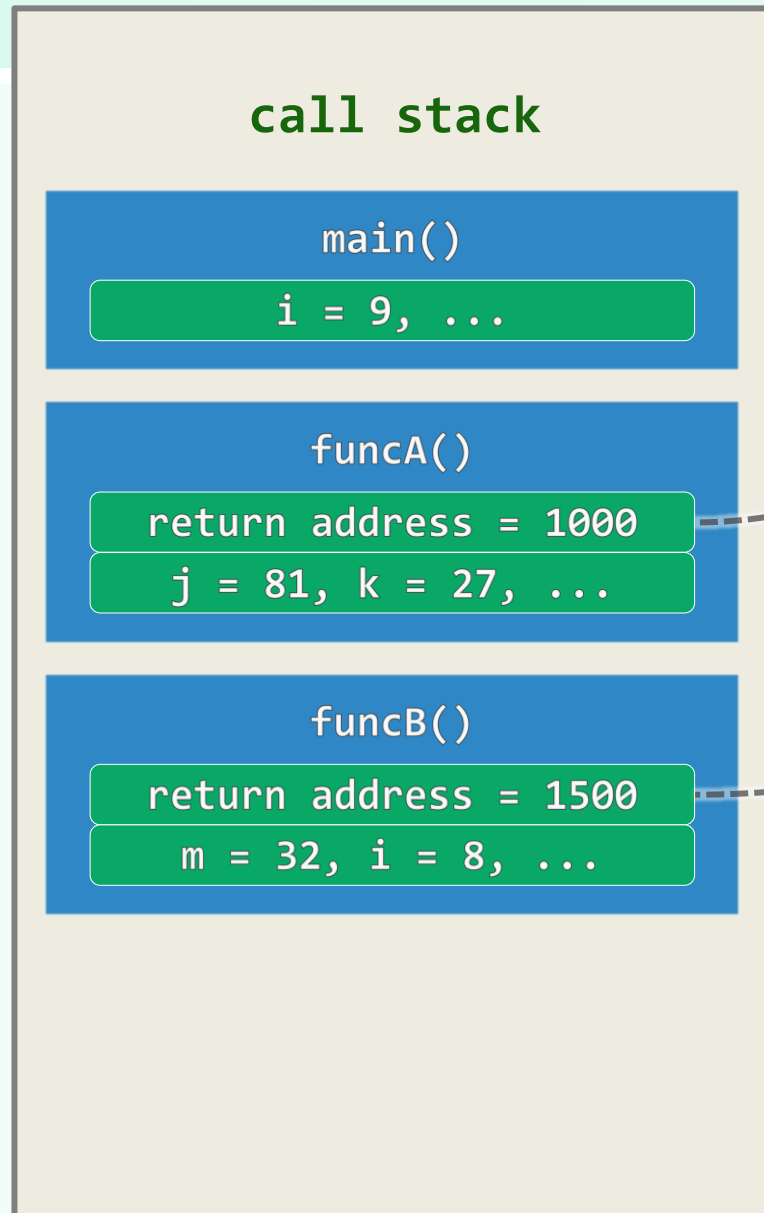
main()

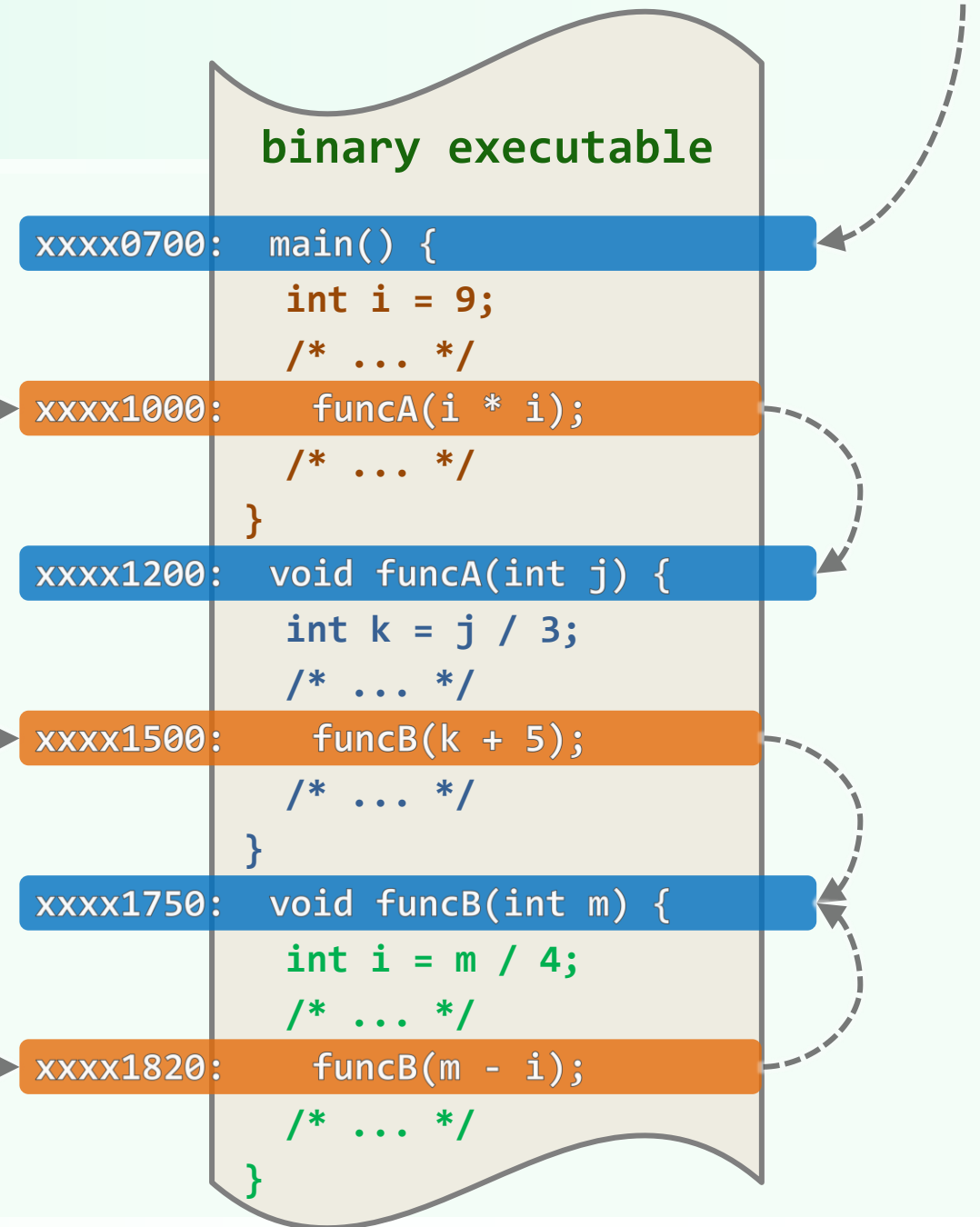
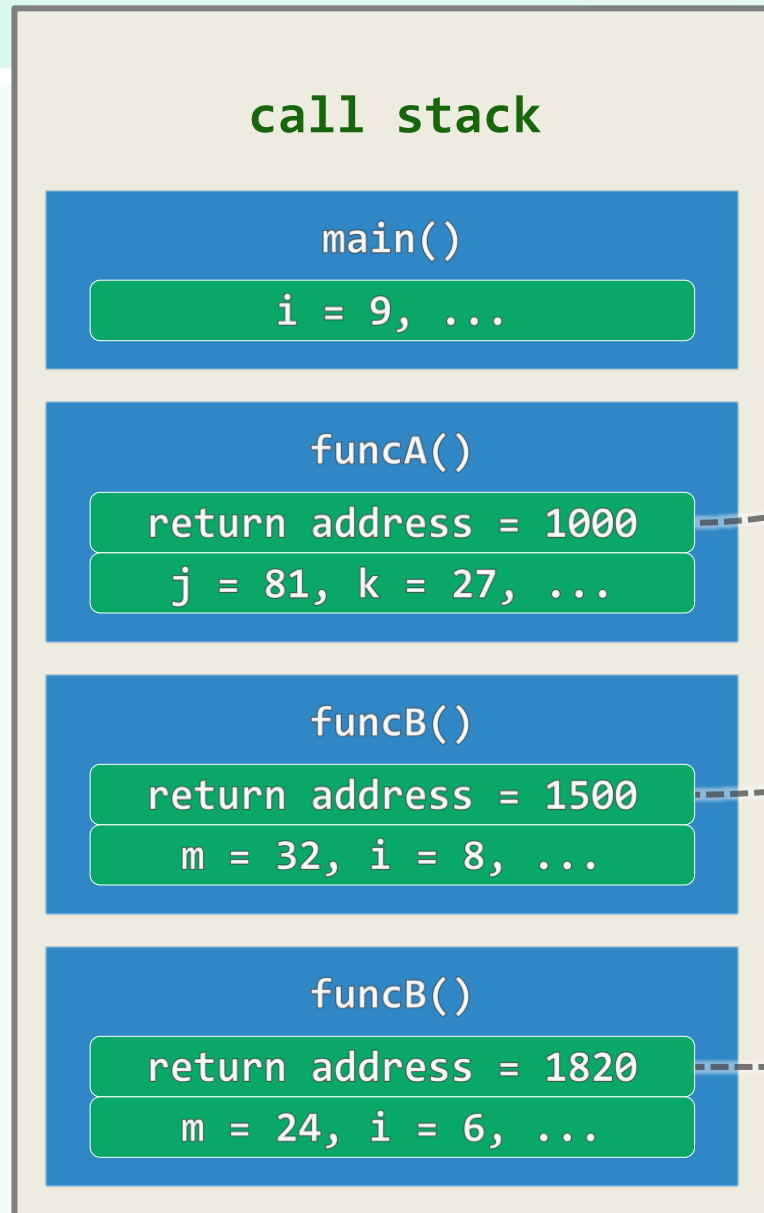
i = 9, ...

binary executable

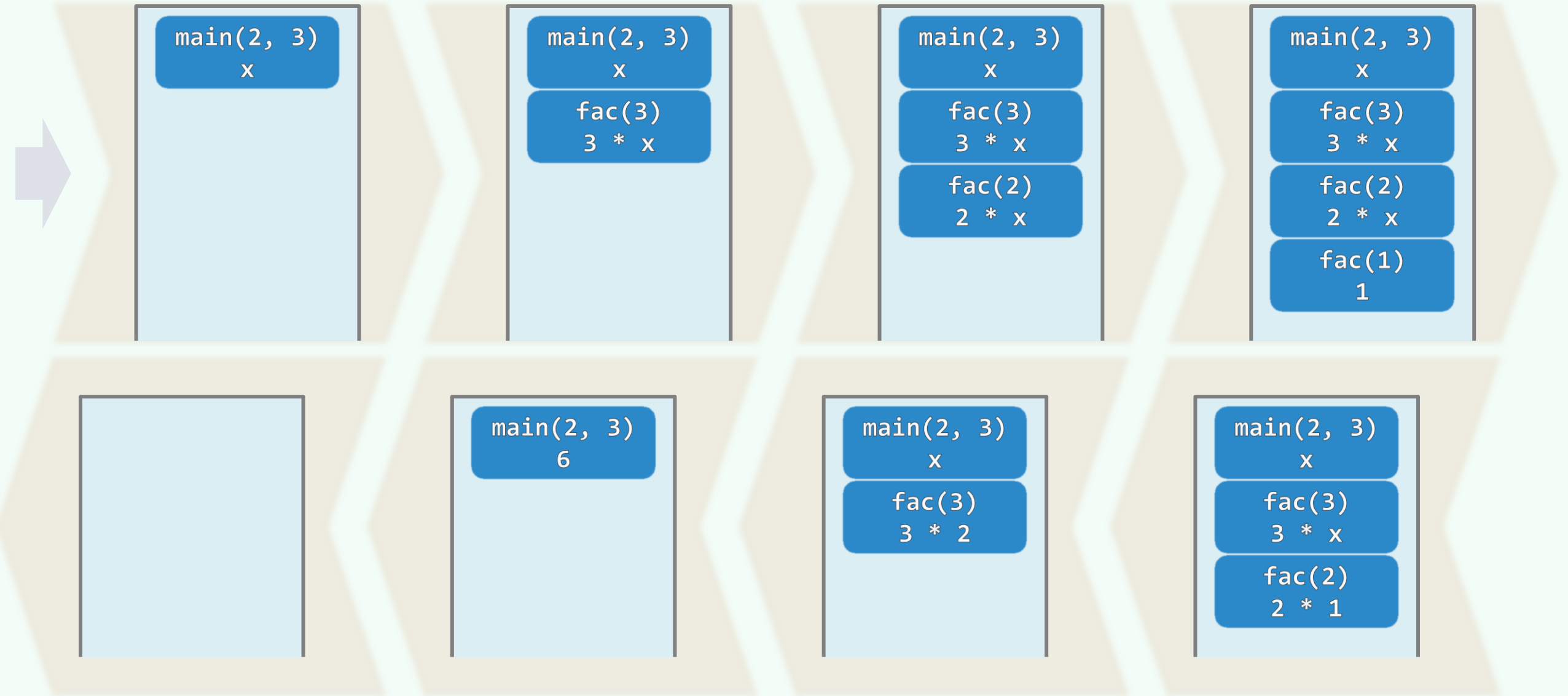
```
xxxx0700: main() {  
           int i = 9;  
           /* ... */  
xxxx1000:   funcA(i * i);  
           /* ... */  
           }  
xxxx1200: void funcA(int j) {  
           int k = j / 3;  
           /* ... */  
xxxx1500:   funcB(k + 5);  
           /* ... */  
           }  
xxxx1750: void funcB(int m) {  
           int i = m / 4;  
           /* ... */  
xxxx1820:   funcB(m - i);  
           /* ... */  
           }
```



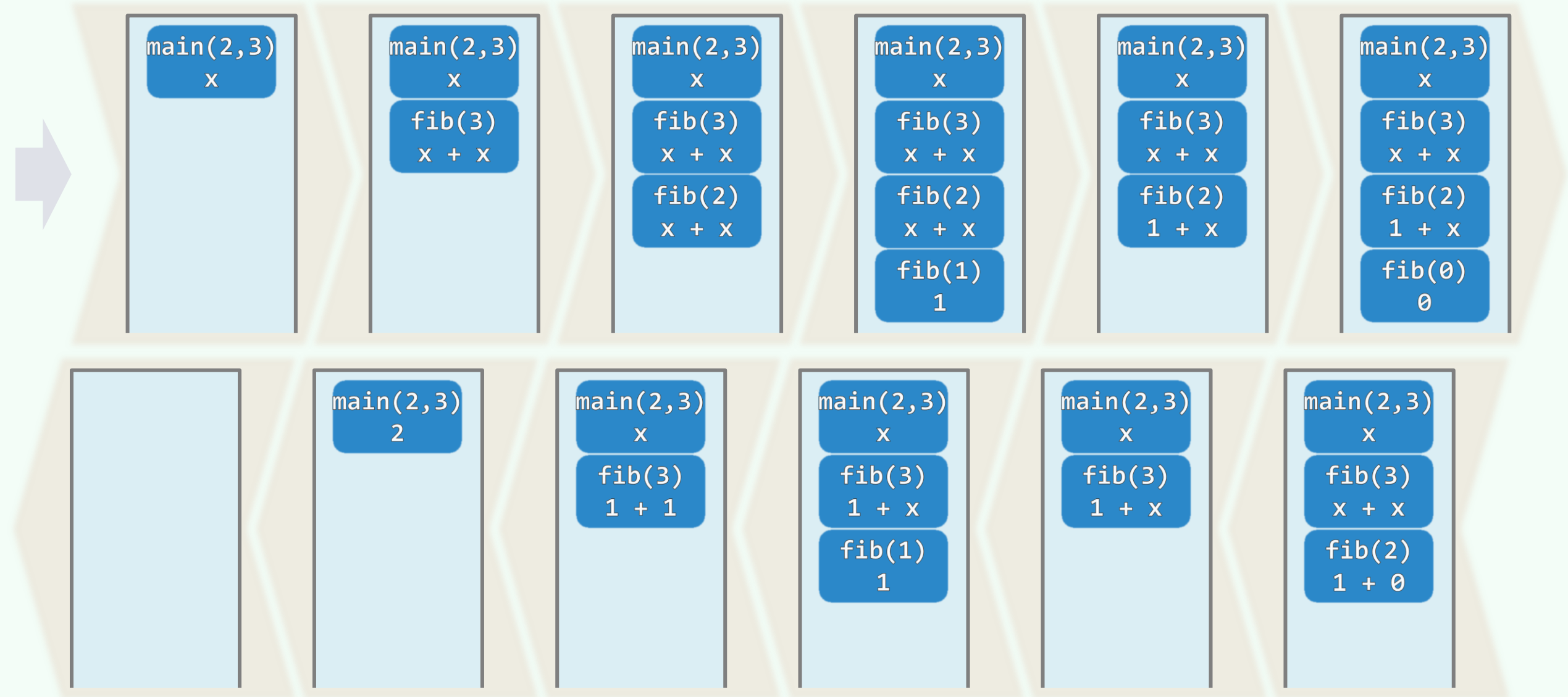




```
int fac(int n) { return (n < 2) ? 1 : n * fac(n - 1); }
```



```
int fib( int n ) { return (n < 2) ? n : fib(n - 1) + fib(n - 2); }
```



空间复杂度

- ❖ `hailstone(int n) {
 if (1 < n)
 n % 2 ? odd(n) : even(n);
}`
- ❖ `even( int n ) { hailstone( n / 2 ); }`
`odd( int n ) { hailstone( 3*n + 1 ); }`
- ❖ `main(int argc, char* argv[])
 { hailstone(atoi(argv[1])); }`
- ❖ 可见，递归算法所需的**空间**
主要取决于递归**深度**，而非递归实例总数

call stack

main(2, 10)
hailstone(10)
even(10)
hailstone(5)
odd(5)
hailstone(16)
even(16)
hailstone(8)
even(8)
hailstone(4)
even(4)
hailstone(2)
even(2)
hailstone(1)

call stack

main(2, 27)
hailstone(27)
odd(27)
hailstone(82)
even(82)
hailstone(41)
odd(41)
hailstone(124)
even(124)
hailstone(62)
even(62)
hailstone(31)
odd(31)
hailstone(94)
... ..

04-B2

栈与队列

调用栈：尾递归

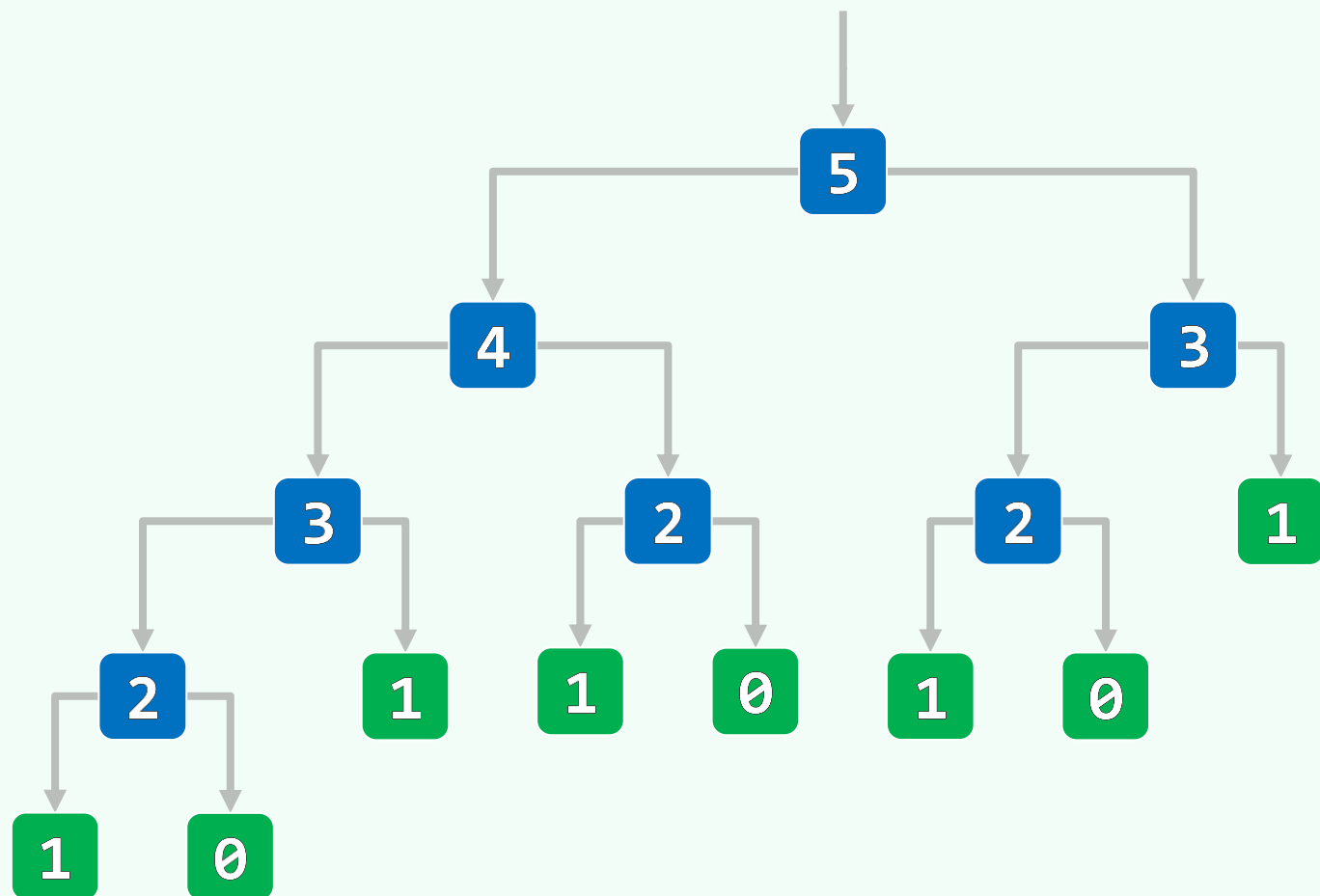
無垂不縮，無往不收

《星期评论》问我“女子解放从那里做起？”
我的答案是：“女子解放当从女子解放做起。此外更无别法。”

邓俊辉

deng@tsinghua.edu.cn

消除递归：动机 + 方法



❖ 递归函数的空间复杂度

- 主要取决于**递归深度**
- 而非递归**实例总数**

❖ 为**隐式地**维护调用栈

需花费额外的时间、空间

❖ 为节省空间，可

- **显式地**维护调用栈
- 将递归算法改写为迭代版本...

消除递归：实例

❖ 通常，消除递归只是在**常数**意义上优化空间

❖ 但也可能有**实质**改进

...

❖ `int fac( int n ) {`
 `int f = 1; //O(1)空间`
 `while ( n > 1 )`
 `f *= n--;`
 `return f;`
}

❖ `void hailstone( int n ) { //O(1)空间`
 `while ( 1 < n )`
 `n = n % 2 ? 3*n + 1 : n/2;`
}

❖ `int fib( int n ) { //O(1)空间`
 `int f = 0, g = 1;`
 `while ( 0 < n-- )`
 `{ g += f; f = g - f; }`
 `return f;`
}

尾递归：定义 + 实例

❖ **线性递归**：函数至多自我调用**一次**

❖ **尾递归**：线性递归的特例

唯一的那句递归恰好位于函数**末尾** (return之前)

❖ 比如，此前学过的...

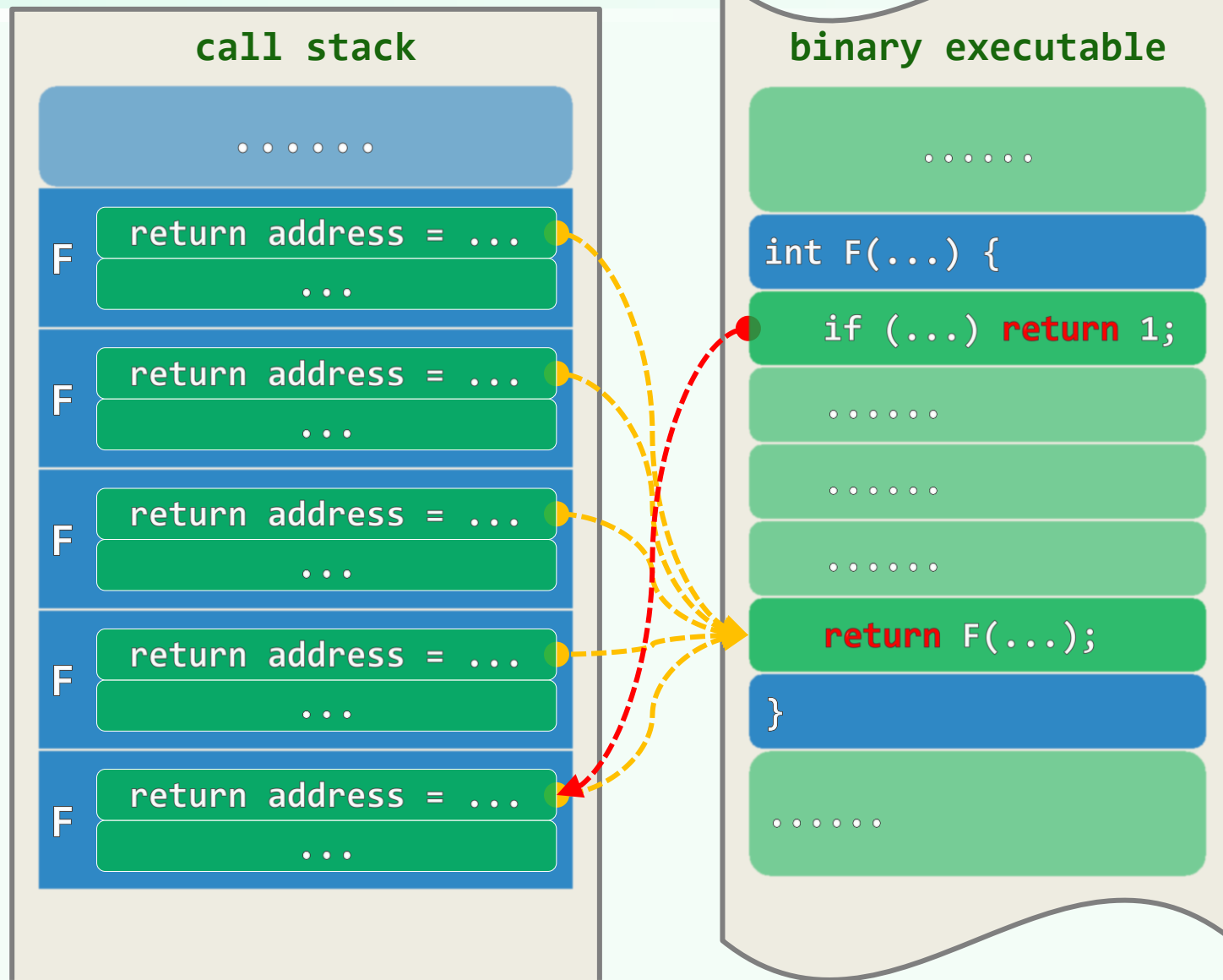
```
void reverse( int * A, int n ) {  
    if ( n < 2 ) return;  
    swap( A[0], A[n - 1] );  
    reverse( A + 1, n - 2 );  
}
```

❖ **形式简单**，实则**多余**...



尾递归：性质

- ❖ 一旦抵达递归基，便会
 - 触发一连串的return
(且返回地址相同)
 - 调用栈连续pop，瞬间清空
- ❖ 每有新帧入栈，老帧随即废弃
何不直接复写老帧？！
- ❖ 越来越多的编译器可以
自动识别并做尾调优化 | TCO
- ❖ 时间复杂度有常系数改进
空间复杂度或有渐近改进



尾递归：消除

//尾递归

```
reverse(int* A, int n) {
```

```
    if (n < 2) return;
```

```
    swap(A[0], A[n-1]);
```

```
    A++; n -= 2;
```

```
    reverse(A, n);
```

```
} //O(n)时间 + O(n)空间
```

//统一转换为迭代

```
reverse(int* A, int n) {
```

```
    next: //转向标志
```

```
    if (n < 2) return;
```

```
    swap(A[0], A[n-1]);
```

```
    A++; n -= 2;
```

```
    goto next; //模拟递归返回
```

```
} //O(n)时间 + O(1)空间
```

//进一步规整化

```
reverse(int* A, int n) {
```

```
    while (1 < n) {
```

```
        swap(A[0], A[n-1]);
```

```
        A++; n -= 2;
```

```
    }
```

```
} //O(n)时间 + O(1)空间
```

栈与队列

调用栈：头递归

Hickory, Dickory, Dock
The mouse ran up the clock

wan, twan, tetbera, metbera, pimp,
setbera, letbera, bovera, dovera, dick,
wanadick, twanadick, tetberadick, metberadick, pimpdick,
setberadick, letberadick, boveradick, doveradick, bumfit,
wanabumfit, ...

邓俊辉

deng@tsinghua.edu.cn

进制转换

❖ 给定任一10进制非负整数，将其转换为 λ 进制表示形式

- $12345_{(10)} = 30071_{(8)}$

- `printf( "%d | %I64d | %b | %o | %x" , n );`

❖ 巴比伦楔形文字 (Babylonian cuneiform) 中的60进制...

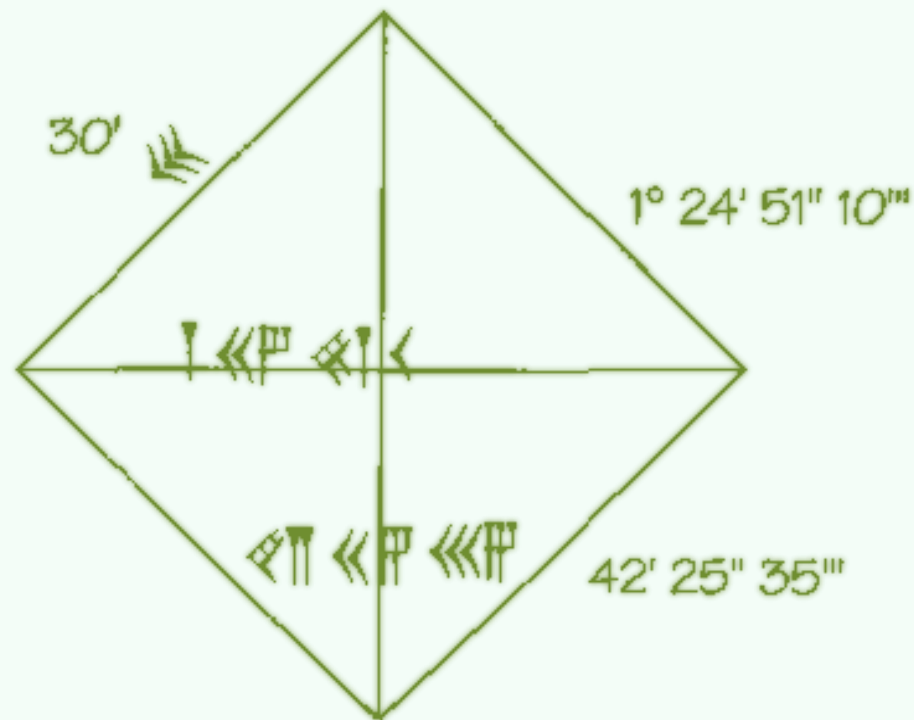
❖ $1^{\circ}24'51''10'''$

$$= 1 + 24/60 + 51/60^2 + 10/60^3$$

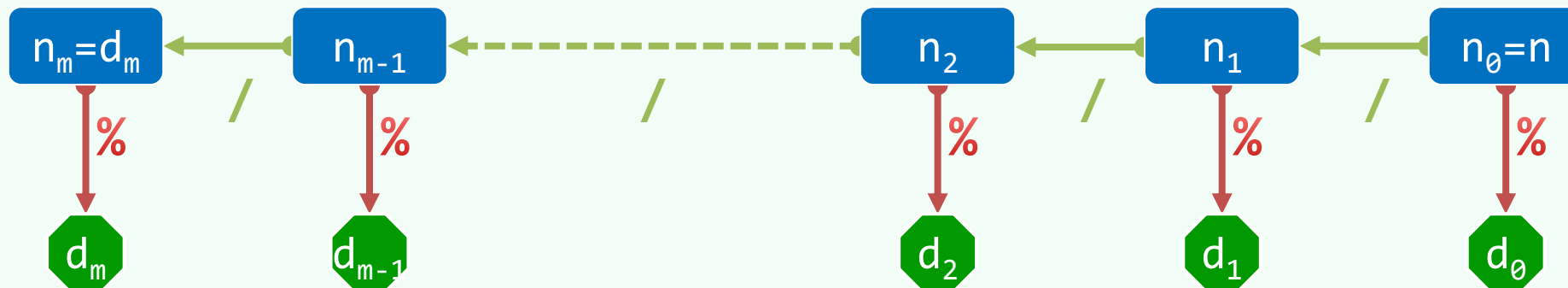
$$= 1.41421296\underline{296296296}\dots \quad // \text{正方形的对角线}$$

❖ 误差 $\sim |1^{\circ}24'51''10''' - \sqrt{2}|$

$$< 0.000,000,6 = 0.6 \times 10^{-6} \quad // \text{即便边长为1km, 误差亦不足1mm}$$



短除法：整商 + 余数

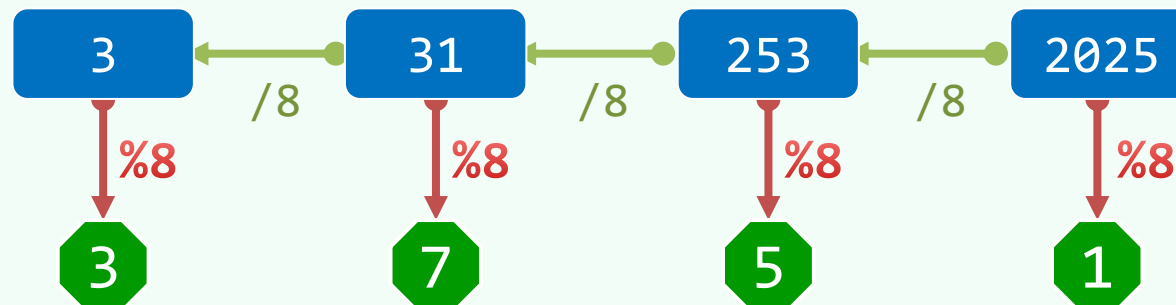


❖ 设: $n = (d_m \dots d_2 d_1 d_0)_\lambda = d_m \cdot \lambda^m + \dots + d_2 \cdot \lambda^2 + d_1 \cdot \lambda^1 + d_0 \cdot \lambda^0$

令: $n_i = (d_m \dots d_{i+2} d_{i+1} d_i)_\lambda$

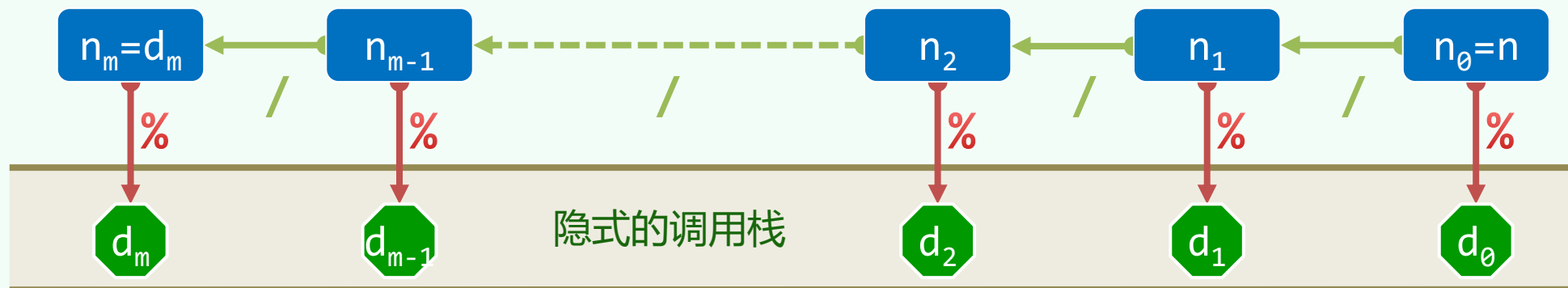
❖ 回意短除法，就是...反复推算

$n_{i+1} = n_i / \lambda$ 和 $d_i = n_i \% \lambda$



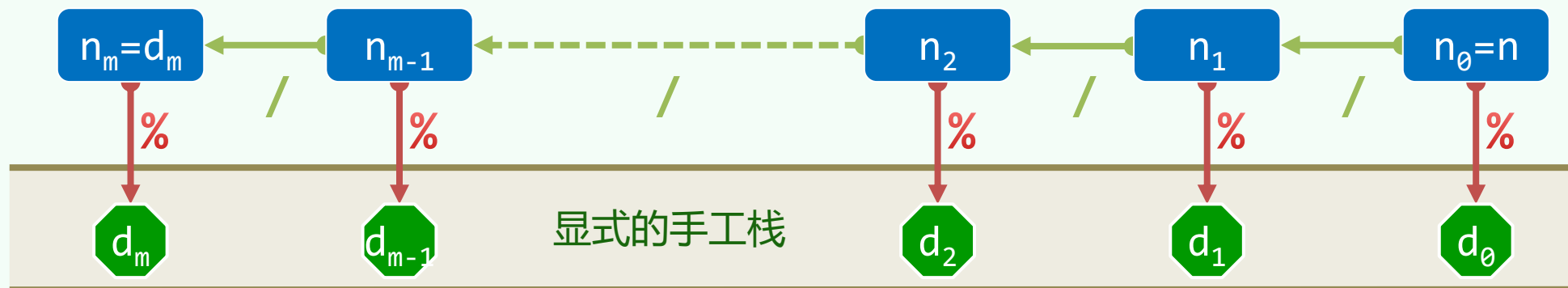
❖ 如此对 λ 反复整除、模余，即可自低而高得出 λ 进制的各位

递归版 | 头递归



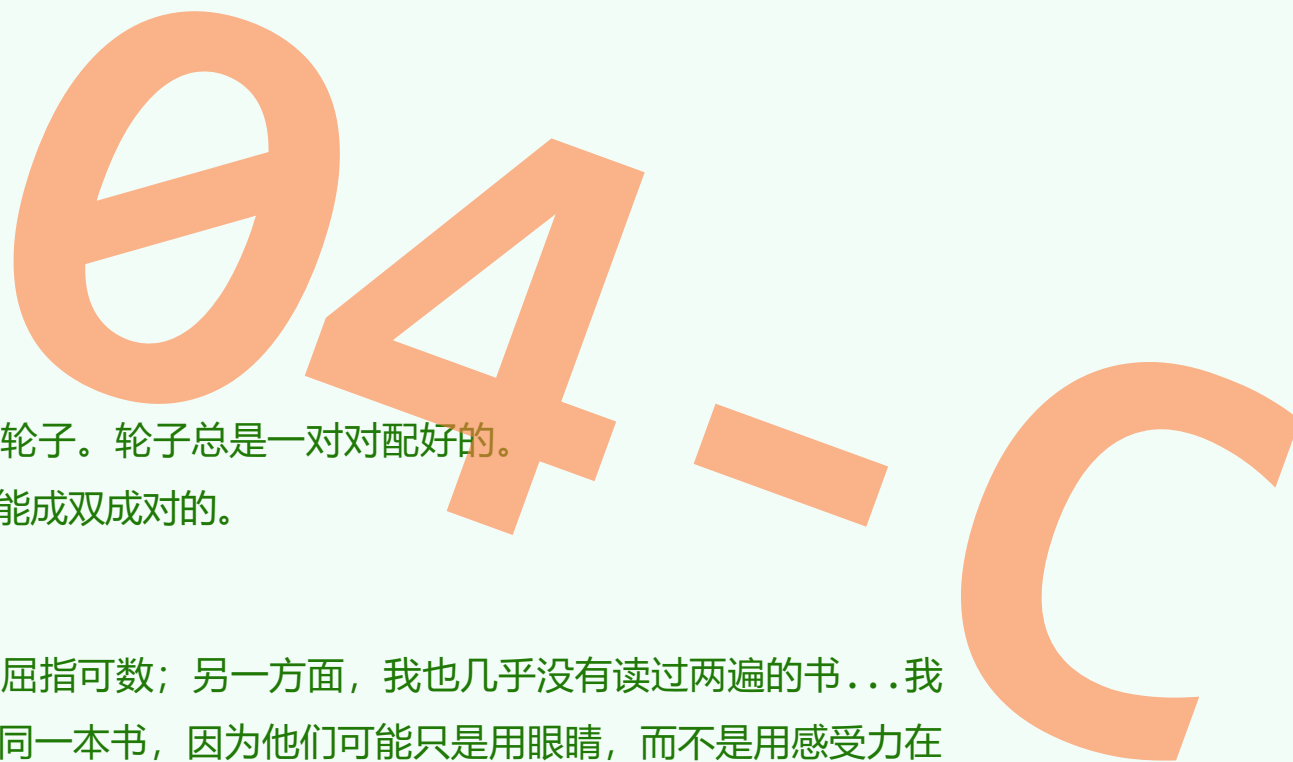
```
❖ void convert( uint64_t n, int base ) {  
    static char digit[] = "0123456789ABCDEF"; // 数位符号，如有必要可相应扩充  
    if ( n < 1 ) return; // 递归基  
    convert( n / base, base ); // 头递归：先输出所有的高位  
    output( digit[n % base] ); // 再输出最低位（具体方式可自定义）  
} // 时间、空间均为 $O(\log n)$ ，但常数相对于迭代版更大
```

迭代版 | 手工栈



```
❖ void convert( uint64_t n, int base ) {  
    static char digit[] = "0123456789ABCDEF";  
    Stack<char> S; //代替调用栈  
    while ( 0 < n ) { S.push( digit[ n % base ] ); n /= base; } //模余入栈, 除商更新  
    while ( !S.empty() ) output( S.pop() ); //按常规次序输出各数位  
}
```

//时间、空间均为 $O(\log n)$, 但**常数**相对于递归版**更小**



栈与队列

括号匹配

我没有替您这辆车准备好轮子。轮子总是一对对配好的。
两只轮子不是偶然碰上就能成双成对的。

我没有从头到尾读完的书屈指可数；另一方面，我也几乎没有读过两遍的书...我知道有人一遍又一遍地读同一本书，因为他们可能只是用眼睛，而不是用感受力在阅读...这样做固然没有害处，但如果他们认为这是一种智力活动，那就错了

邓俊辉

deng@tsinghua.edu.cn

括号匹配

❖ 匹配与失配

$(a[b(i)-1][c(j)+1] + d[e(i)+f(j)][e(i)-f(j)])*(i+j)$ //匹配

$(a[b(i)-1][c(j)+1]) + d[e(i)+f(j)][e(i)-f(j)]*(i+j)$ //失配

❖ 忽略非括号字符

$([()] [()] [() ()] [() ()]) ()$ //匹配

$([()] [()]) [() ()] [() ()]) ()$ //失配

❖ 先考虑一种括号

$((()) (()) (() ()) (() ())) ()$ //匹配

$((()) (())) (() ()) (() ())) ()$ //失配

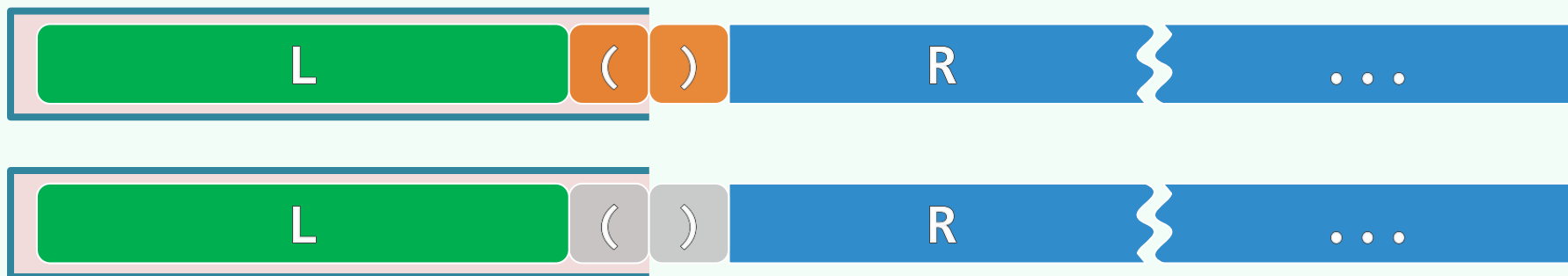
由内而外，减而治之：掏洋葱

❖ 原理：消去一对**紧邻**的成对括号，原表达式匹配与否不受影响



❖ 事实：每棵洋葱都有**芯** | kernel；且一趟**扫描**便可找到——但简单地照此迭代，却要 $O(n^2)$ 时间

关键：如何**高效**地找到，从而保证减治得以**持续**进行？



❖ 只扫一趟, 绝不回头：用栈**记录**已经扫到、但尚待处理的部分（其实全是**左括号**）

遇 '**(**' 进栈；遇到 '**)**'，则出栈，并**掏芯**

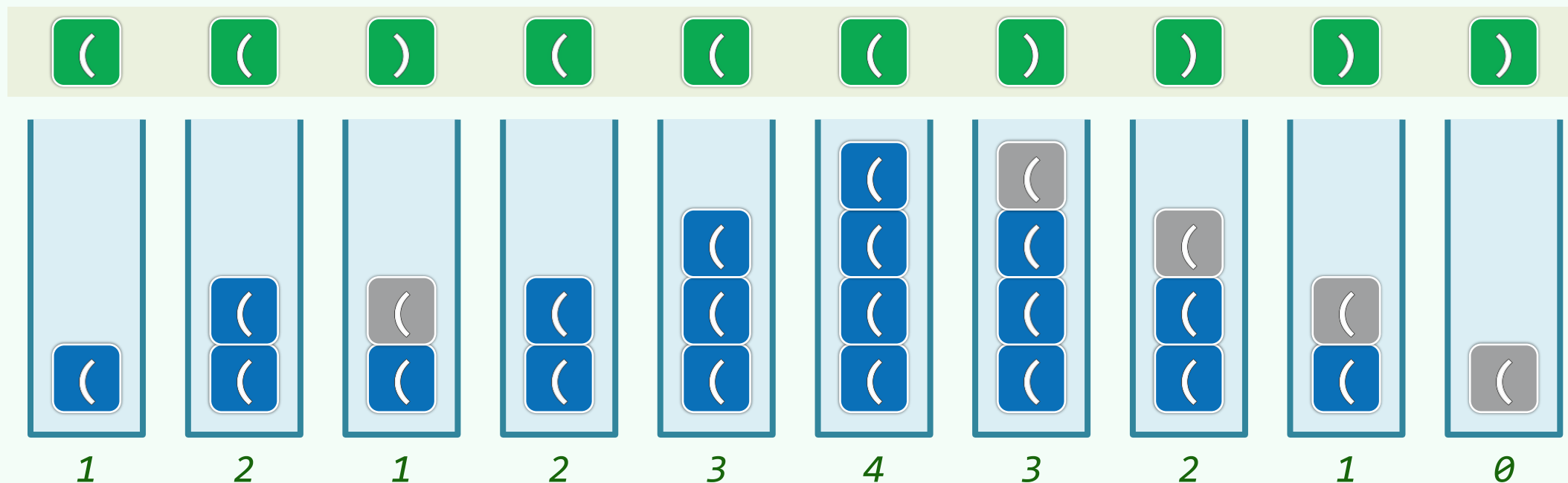
单种括号：算法

```
bool paren( const char A[], Rank n ) {  
  
    Stack<char> S; //记录已发现，但尚未匹配的左括号  
  
    for ( Rank i = 0; i < n; i++ ) //一趟顺序扫描  
  
        if ( '(' == A[i] ) S.push( A[i] ); //遇左括号：则进栈  
  
        else if ( ! S.empty() ) S.pop(); //遇右括号：若栈非空，则弹出对应的左括号  
  
        else return false; //否则（左括号缺失）  
  
    return S.empty(); //最终栈空，当且仅当匹配  
  
}
```

单种括号：简化

❖ 实际上，若仅考虑一种括号，只需一个**计数器**足矣：`s = S.size()`

左括号则**增**，右括号则**减**；一旦转**负**，则为失配；最后归**零**，即为匹配；否则也是失配



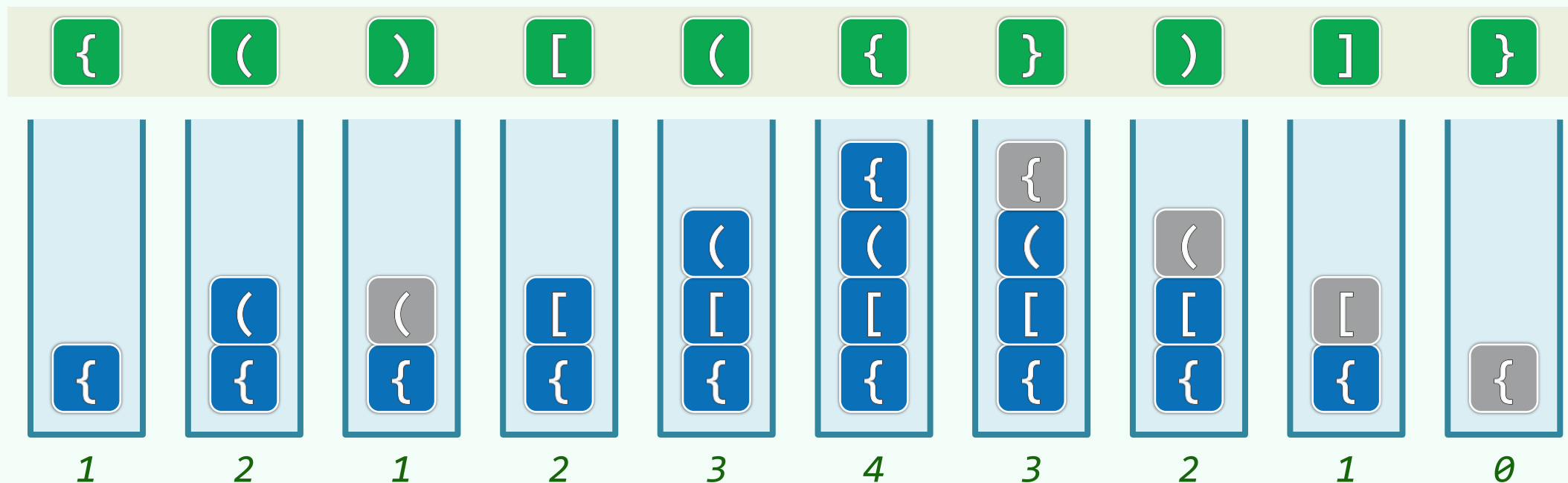
❖ 很遗憾，当有**多种**括号并存时，即使用**多个**计数器，仍可能**误报**



多类括号：还得是栈

❖ 还须回头求助于栈：一个栈，便足以应对多类括号

左括号一律进栈；右括号的处置更加精细：必须与栈顶左括号属于同类



❖ 甚至，可以推广至XML等格式：它们支持自定义元素 | custom elements，可自定义无数种括号

多类括号：算法

```
bool paren( const char A[], Rank n ) {  
    Stack<char> S; //无论有多少种括号，依然只需一个栈  
    for ( Rank i = 0; i < n; i++ ) //还是一趟顺序扫描  
        switch ( A[i] ) { //左括号无论何种，一律进栈；右括号必须与栈顶左括号同类  
            case '(': case '[': case '{': S.push( A[i] ); break;  
            case ')': if ( S.empty() || '(' != S.pop() ) return false; break;  
            case ']': if ( S.empty() || '[' != S.pop() ) return false; break;  
            case '}': if ( S.empty() || '{' != S.pop() ) return false; break;  
        }  
    return S.empty(); //最终栈空，当且仅当匹配  
}
```

04-D

栈与队列

表达式求值

邓俊辉

deng@tsinghua.edu.cn

知實而不知名，知名而不知實，皆不知也

应用

❖ 给定语法正确的算术表达式s，计算与之对应的数值

❖ UNIX: \$ **echo \$((0 + (1 + 23) / 4 * 5 * 67 - 8 + 9))**

❖ DOS: \> **set /a (!0 ^<^< (1 - 2 + 3 * 4)) - 5 * (6 ^| 7) / (8 ^^ 9)**

❖ PS: GS> 0 1 23 add 4 div 5 mul 67 mul add 8 sub 9 add =

❖ Excel: = COS(0) + 1 - (2 - POWER((FACT(3) - 4), 5)) * 67 - 8 + 9

❖ Word: = NOT(0) + 12 + 34 * 56 + 7 + 89

❖ calc: 0 ! + 12 + 34 * 56 + 7 + 89 =

❖ calc: 0 ! + 1 - (2 - (3 ! - 4) y 5) * 67 - 8 + 9 =

减而治之

❖ 优先级：约定俗成 + 括号

❖ 优先级高的局部

- 执行计算并
- 以其数值替代

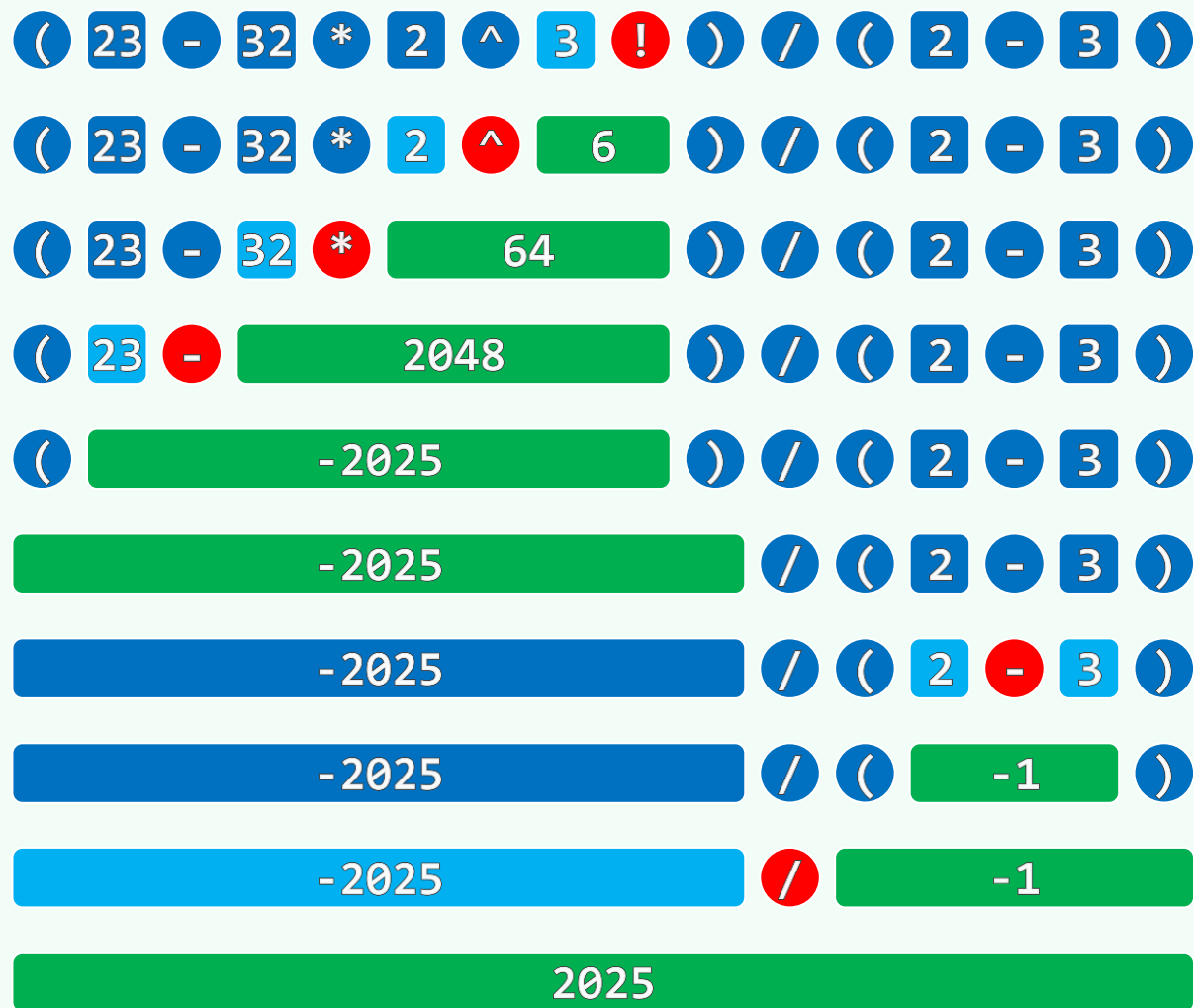
c)

❖ 洋葱芯：一趟顺序扫描即可找到

❖ 如何保证始终都能高效查找

从而减治得以持续推进？

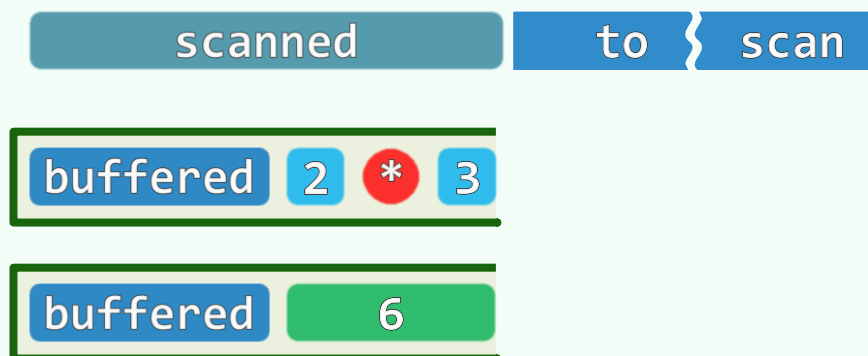
❖ 还是要求助于栈...



以栈缓冲：一趟扫描，绝不回头 (1/2)

❖ 已扫到、但尚待处置的部分用**栈**缓存

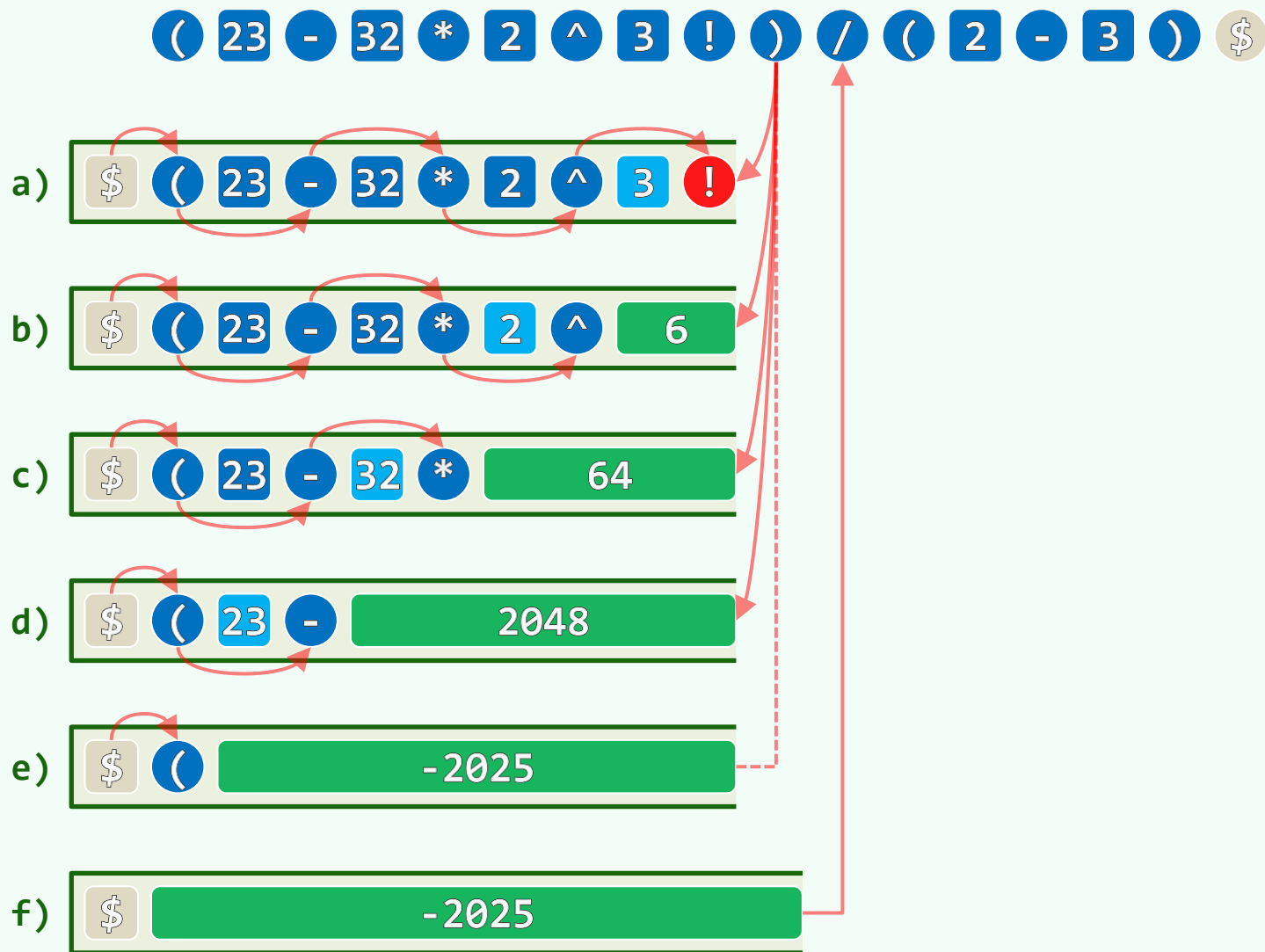
计算后，代以中间结果



❖ 任何时刻，栈内运算符的优先级

必然自底向顶，**渐次升高**

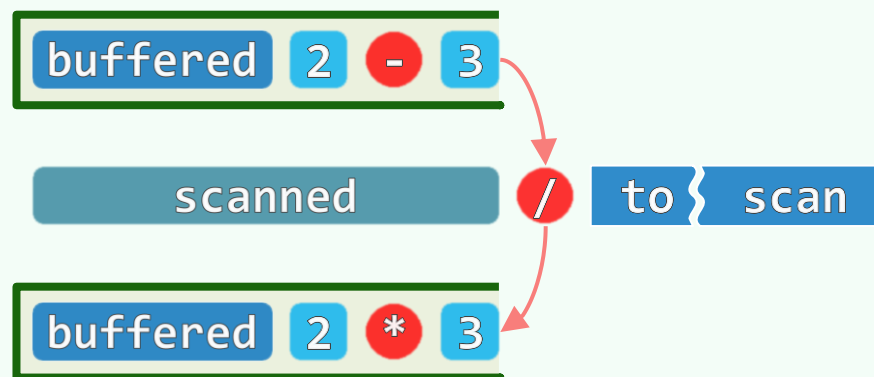
于是，栈顶部分能否计算...



以栈缓冲：一趟扫描，绝不回头 (2/2)

❖ ...只取决于**局部**的相对优先级

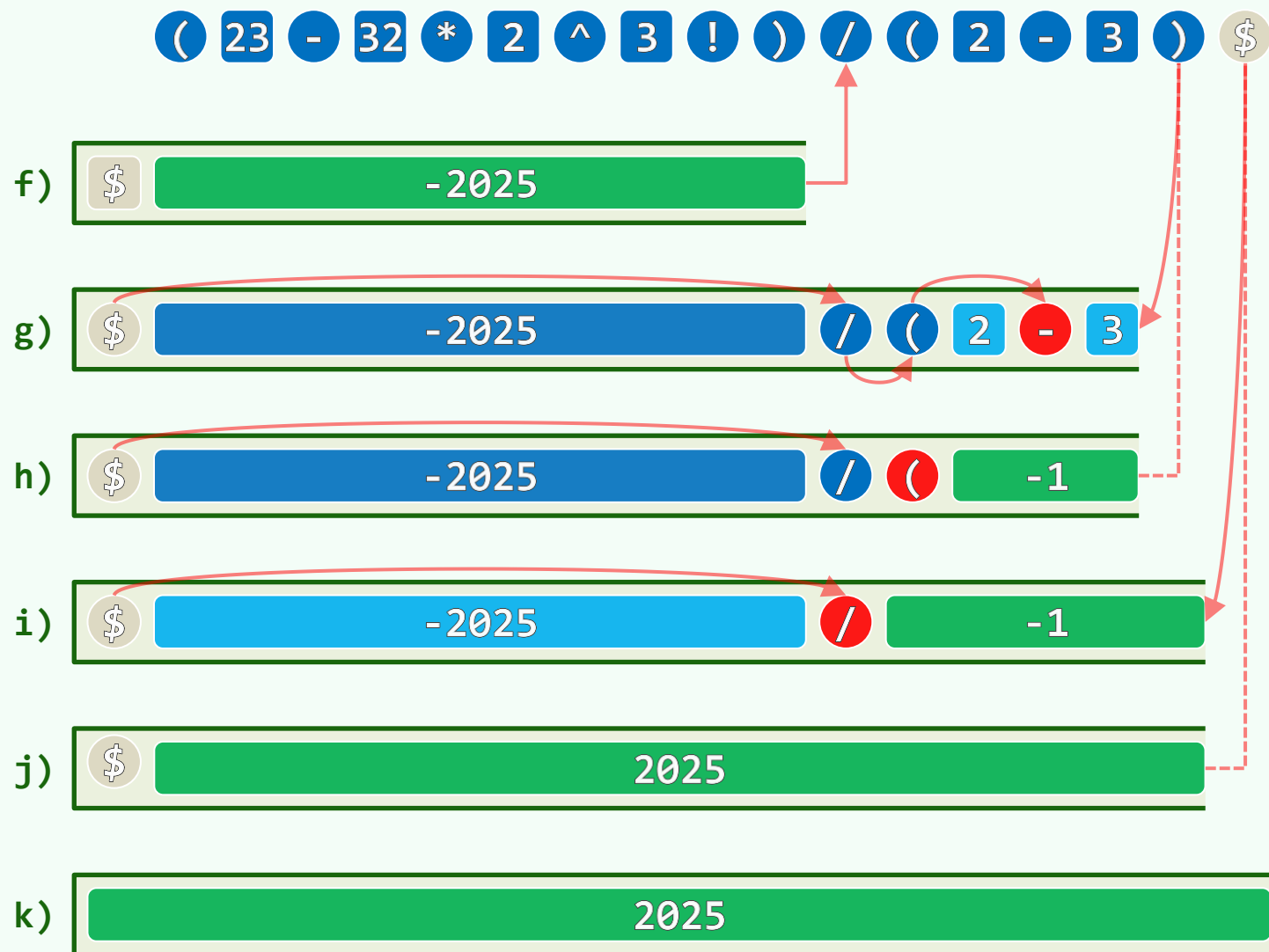
栈顶运算符 ~ **当前运算符**



❖ 相对优先级：如何快捷地查询、比较？

数、符混居，多有不便，如何消除？

括号、哨兵，如何统一处理？



优先级表

```
const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]

    /* -- + */  '>', '>', '<', '<', '<', '<', '<', '>', '>',
    /* | - */  '>', '>', '<', '<', '<', '<', '<', '>', '>',
    /* 栈 * */  '>', '>', '>', '>', '<', '<', '<', '>', '>',
    /* 顶 / */  '>', '>', '>', '>', '<', '<', '<', '>', '>',
    /* 运 ^ */  '>', '>', '>', '>', '>', '<', '<', '>', '>',
    /* 算 ! */  '>', '>', '>', '>', '>', '>', ' ', '>', '>',
    /* 符 ( */  '<', '<', '<', '<', '<', '<', '<', '~', ' ',
    /* | ) */  ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', //padding
    /* -- \0 */  '<', '<', '<', '<', '<', '<', '<', ' ', '~'

    //      +      -      *      /      ^      !      (      )      \0
    //      |----- 当前运算符 -----|
```

};

主算法

```
double evaluate( char* S, char* RPN ) { //S保证语法正确

    Stack<double> opnd; Stack<char> optr; //运算数栈、运算符栈

    for ( optr.push(' \0'); !optr.empty(); ) //逐个处理各字符，直至运算符栈空

        if ( isdigit( *S ) ) //若为运算数（可能多位、小数），则
            readNumber( S, opnd ); //读入

        else //若为运算符，则视其与栈顶运算符之间优先级的高低
            switch( prio( optr.top(), *S ) ) { /* 分别处理 */ }

    return opnd.pop(); //弹出并返回最后的计算结果

} //evaluate
```

'<': 推迟计算, 静待时机

```
switch( prio( optr.top(), *S ) ) {
```

```
    case '<':
```

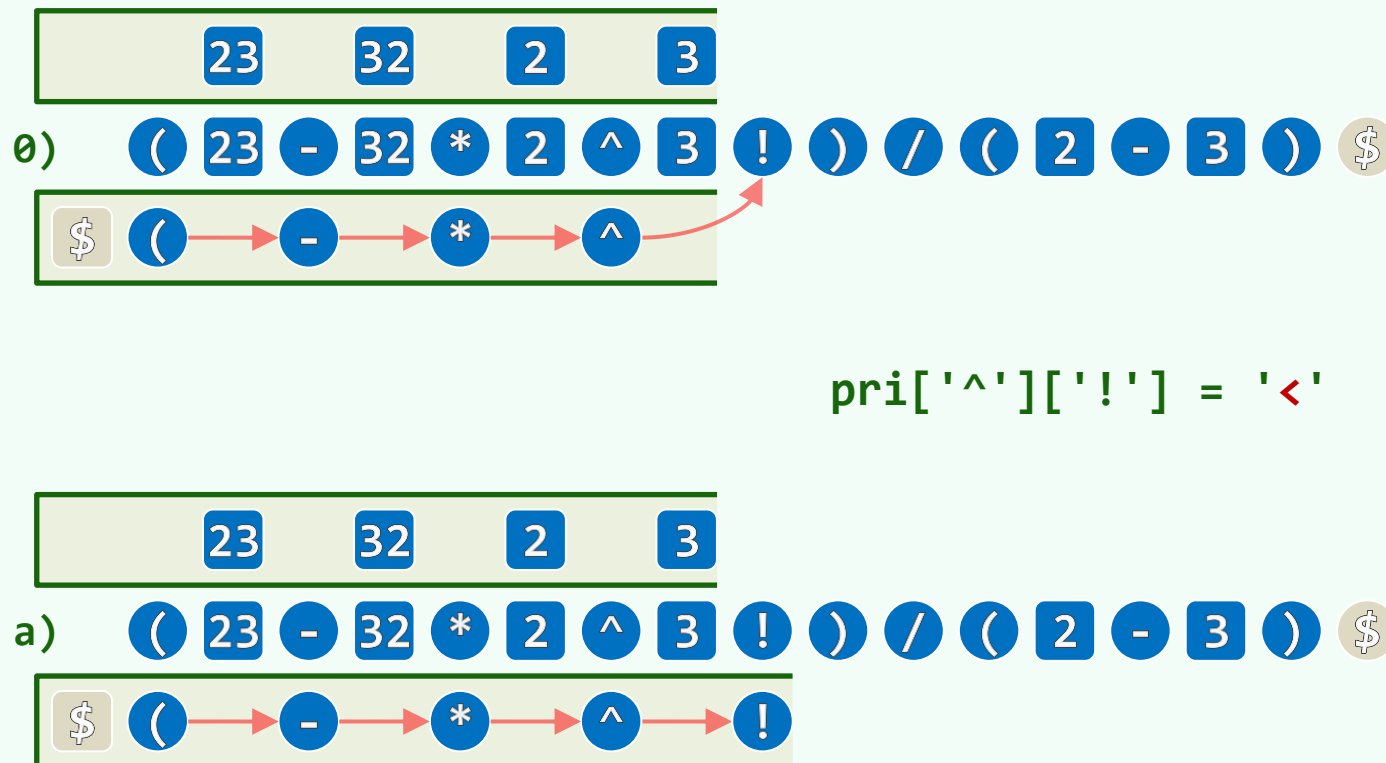
```
        optr.push( *S );
```

```
        S++;
```

```
        break;
```

```
    /* ..... */
```

```
} //switch
```



'>': 时机已到, 执行计算

```
switch( prio( optr.top(), *s ) ) {
```

```
/* ..... */
```

```
case '>':
```

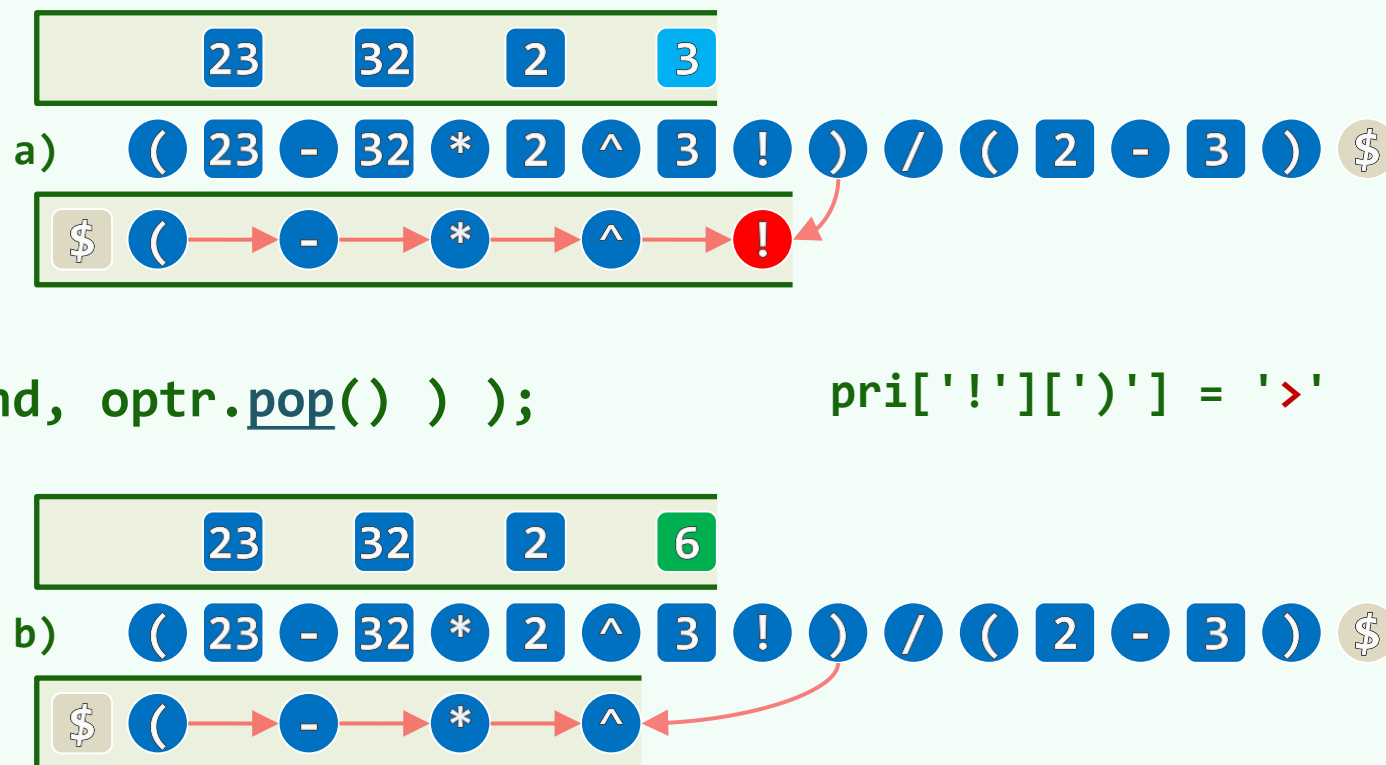
```
    opnd.push( calcu( opnd, optr.pop() ) );
```

```
    pri['!'][')'] = '>'
```

```
break;
```

```
/* ..... */
```

```
} //switch
```



'~': 终须了断，脱去括号，或者哨兵

```
switch( prio( optr.top(), *S ) ) {
```

```
/* ..... */
```

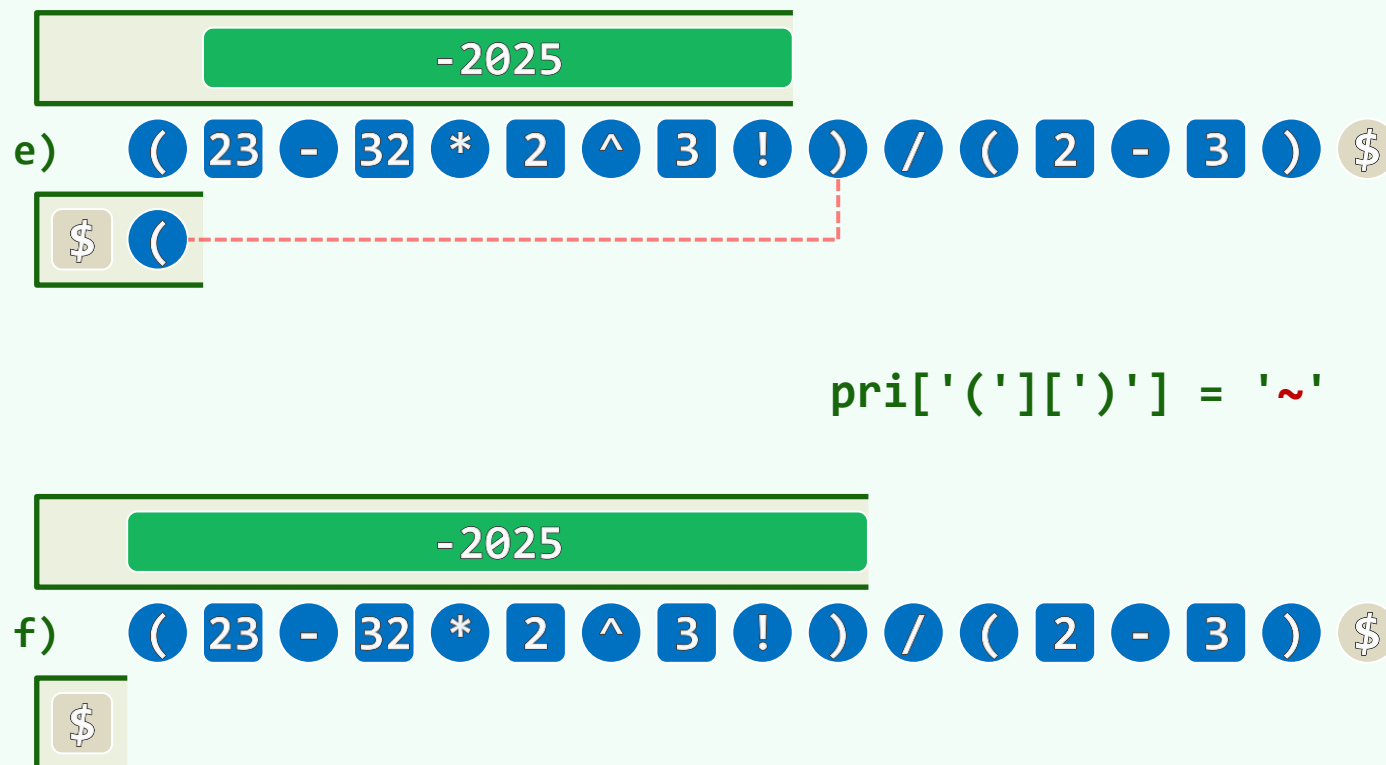
```
case '~':
```

```
    optr.pop();
```

```
    S++;
```

```
    break;
```

```
} //switch
```



```
pri['('][')'] = '~'
```

+ - * / !: 芸芸众生

const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]

/* -- + */	'>', '>', '<', '<', '<', '<', '<', '>', '>',
/* - */	'>', '>', '<', '<', '<', '<', '<', '>', '>',
/* 栈 * */	'>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 顶 / */	'>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 运 ^ */	'>', '>', '>', '>', '>', '<', '<', '>', '>',
/* 算 ! */	'>', '>', '>', '>', '>', '>', '>', '>', '>',
/* 符 (*/	'<', '<', '<', '<', '<', '<', '<', '~', ' ',
/*) */	' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', //padding
/* -- \0 */	'<', '<', '<', '<', '<', '<', '<', ' ', '~'
//	+ - * / ^ ! () \0
//	-----当前运算符-----

'('：我不下地狱，谁下地狱

const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]

/* -- + */	'>', '>', '<', '<', '<', '<', '<', '>', '>',
/* - */	'>', '>', '<', '<', '<', '<', '<', '>', '>',
/* 栈 * */	'>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 顶 / */	'>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 运 ^ */	'>', '>', '>', '>', '>', '<', '<', '>', '>',
/* 算 ! */	'>', '>', '>', '>', '>', '>', ' ', '>', '>',
/* 符 (*/	'<', '<', '<', '<', '<', '<', '<', '~', ' ',
/*) */	' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', //padding
/* -- \0 */	'<', '<', '<', '<', '<', '<', '<', ' ', '~'
//	+ - * / ^ ! () \0
//	-----当前运算符-----

};

') ' : 死线已至 (然后满血复活)

const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]

/* -- + */ '>', '>', '<', '<', '<', '<', '<', '>', '>',

/* | - */ '>', '>', '<', '<', '<', '<', '<', '>', '>',

/* 栈 * */ '>', '>', '>', '>', '<', '<', '<', '>', '>',

/* 顶 / */ '>', '>', '>', '>', '<', '<', '<', '>', '>',

/* 运 ^ */ '>', '>', '>', '>', '>', '<', '<', '>', '>',

/* 算 ! */ '>', '>', '>', '>', '>', '>', ' ', '>', '>',

/* 符 (*/ '<', '<', '<', '<', '<', '<', '<', '~', ' ',

/* |) */ ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', //padding

/* -- \0 */ '<', '<', '<', '<', '<', '<', '<', '<', ' ', '~'

// + - * / ^ ! () \0

// |-----当前运算符-----|

'\0': 从创世纪, 到世界末日

```
const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]
```

Diagram illustrating the state of a stack during the evaluation of the expression `1 + 2 * 3 + 4 * 5 + 6`. The stack is represented by a grid of cells, each containing a character (operator or operand). The stack is initially empty, and the expression is processed from left to right.

The stack is divided into two main sections: the top section (labeled "栈顶") and the bottom section (labeled "运算符").

The top section (栈顶) contains the following characters (from top to bottom):

- Row 1: '>', '>', '<', '<', '<', '<', '<', '>', '>'
- Row 2: '>', '>', '<', '<', '<', '<', '<', '>', '>'
- Row 3: '>', '>', '>', '>', '<', '<', '<', '>', '>'
- Row 4: '>', '>', '>', '>', '<', '<', '<', '>', '>'
- Row 5: '>', '>', '>', '>', '>', '<', '<', '>', '>'
- Row 6: '>', '>', '>', '>', '>', '>', '>', '>', '>'
- Row 7: '<', '<', '<', '<', '<', '<', '<', '~', ''
- Row 8: ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ' (padding)
- Row 9: '<', '<', '<', '<', '<', '<', '<', ' ', '~'

The bottom section (运算符) contains the following characters (from left to right):

- Row 1: '+', '-', '*', '/', '^', '!', '(', ')', '\0

The current operator being processed is indicated by the label "当前运算符" (Current Operator) at the bottom, which points to the character '!' in the bottom section.

The stack is currently full, and the current operator is '!', which is the operator for the next operation to be performed.

04-E

栈与队列

逆波兰表达式

日月逝矣，歲不我與

吾雖不善書，曉書莫如我

知與之為取，政之寶也

巨大的需求源于巨大的财富，而且要得到我们想要的，最好的办法往往是舍弃我们已拥有的

邓俊辉

deng@tsinghua.edu.cn

Reverse Polish Notation

- ❖ 逆波兰表达式: J. Lukasiewicz (1878 ~ 1956)
- ❖ 在由运算符 (operator) 和操作数 (operand) 组成的表达式中
不使用括号 (parenthesis-free), 即可表示带优先级的运算关系
- ❖ 例如: $0 ! + 123 + 4 * (5 * 6 ! + 7 ! / 8) / 9$
 $0 ! 123 + 4 5 6 ! * 7 ! 8 / + * 9 / +$
- ❖ 又如: $(0 ! + 1) ^ (2 * 3 ! + 4 - 5) - 6 ! / (7 + 8 + 9)$
 $0 ! 1 + 2 3 ! * 4 + 5 - ^ 6 ! 7 8 + 9 + / -$
- ❖ 相对于日常使用的中缀式 (infix), RPN亦称作后缀式 (postfix)
- ❖ 作为补偿, 须额外引入一个起分隔作用的**元字符** (比如空格) //较之原表达式, 未必更短

快捷求值

0 ! 123 + 4 5 6 ! * 7 ! 8 / + * 9 / +

```
double rpn( char* S ) { //对RPN求值
```

```
    Stack<double> opnd; //仅需一个运算数栈
```

```
    for ( char* item = strtok( S, " " ); item; item = strtok( NULL, " " ) )
```

```
        if ( 1 < strlen( item ) || isdigit( item[0] ) ) //遇到操作数
```

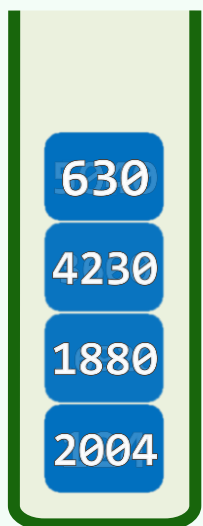
```
            opnd.push( atof( item ) ); //则令其入栈
```

```
        else //遇到运算符
```

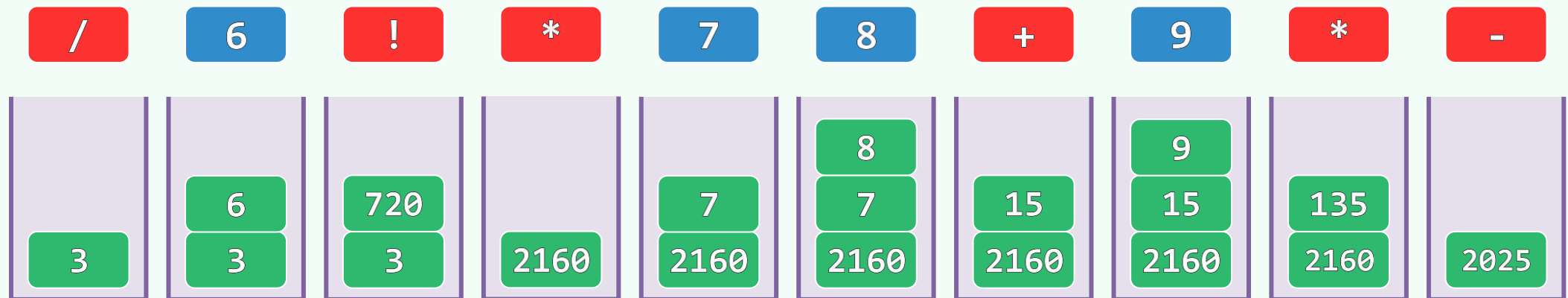
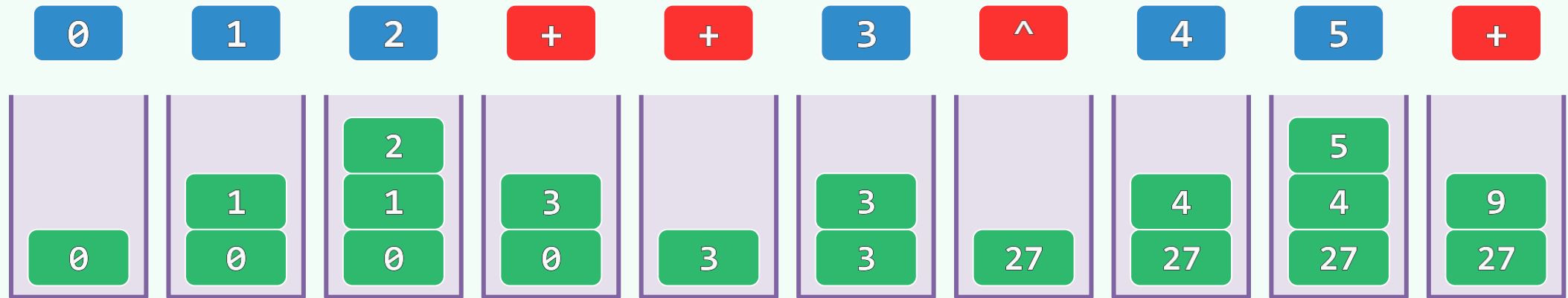
```
            opnd.push( calcu( opnd, item[0] ) ); //则直接计算
```

```
    return opnd.pop(); //唯一尚存的操作数，即为计算结果
```

```
} //rpn
```



0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -



手工转换

a) (0 + (1 + 2)) ^ 3 / (4 + 5) * 6 ! - (7 + 8) * 9

添加括号

b) (((((0 + (1 + 2)) ^ 3) / (4 + 5)) * (6 !)) - ((7 + 8) * 9))

运算符找到各自的右括号

c) (((((0 + (1 + 2)) ^ 3) / (4 + 5)) * (6 !)) - ((7 + 8) * 9))

运算符移至各自的右括号

d) (((((0 + (1 + 2 + + ^ 3 ^ / (4 + 5 + / * (6 ! ! * - ((7 + 8 + * 9 * -

擦除所有左括号

e) 0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -

f) 0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -

自动转换

```
double evaluate( char* S, char* RPN ) { //RPN转换
    /* ..... */
    for ( optr.push(' \0' ); !optr.empty(); )
        if ( isdigit( * S ) ) //若当前字符为操作数，则直接
            { readNumber( S, opnd ); append( RPN, opnd.top() ); } //将其接入RPN
        else //若当前字符为运算符
            switch( prio( optr.top(), *S ) ) {
                /* ..... */
                case '>': //且可立即执行，则在执行相应计算的同时
                    append( RPN, optr.top() ); //将其接入RPN
                /* ..... */
                break;
            }
    /* ..... */
}
```

04-F

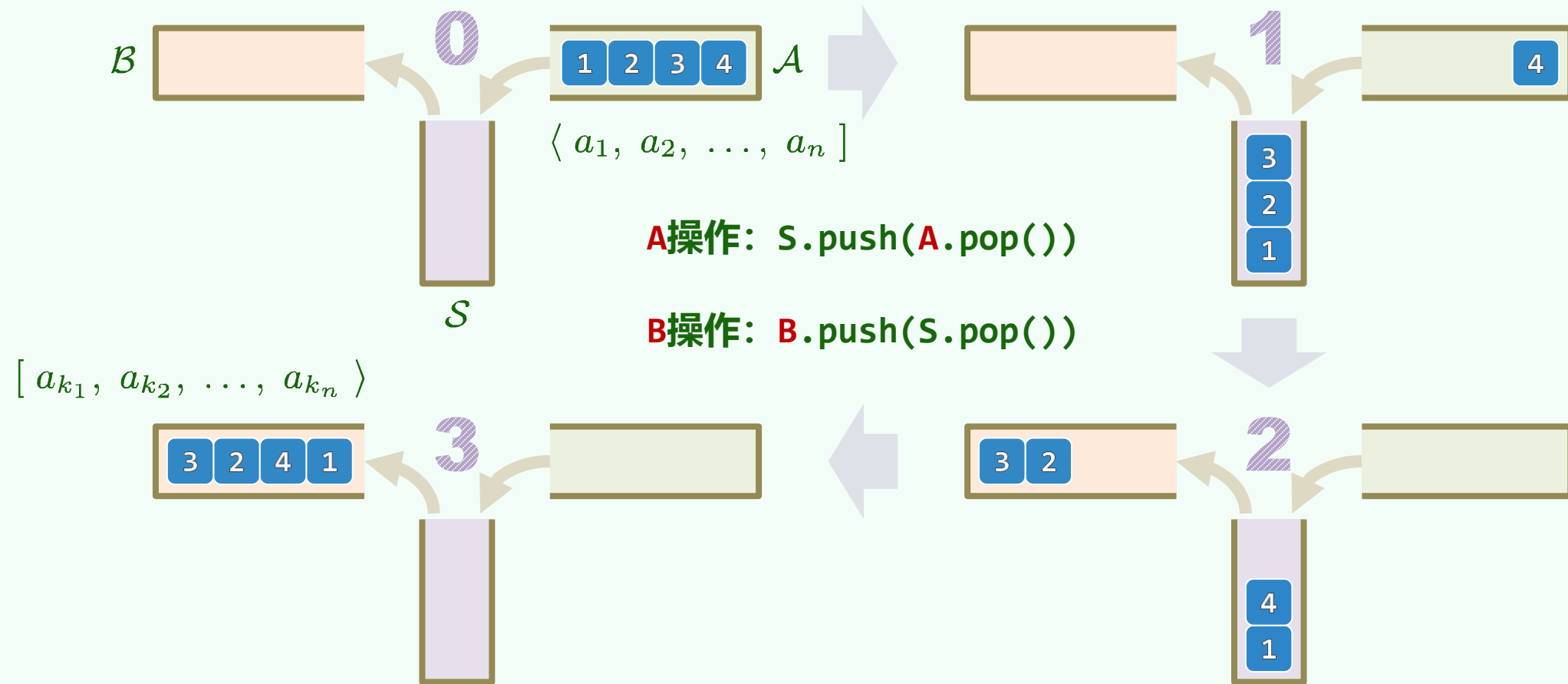
栈与队列

栈混洗

邓俊辉

deng@tsinghua.edu.cn

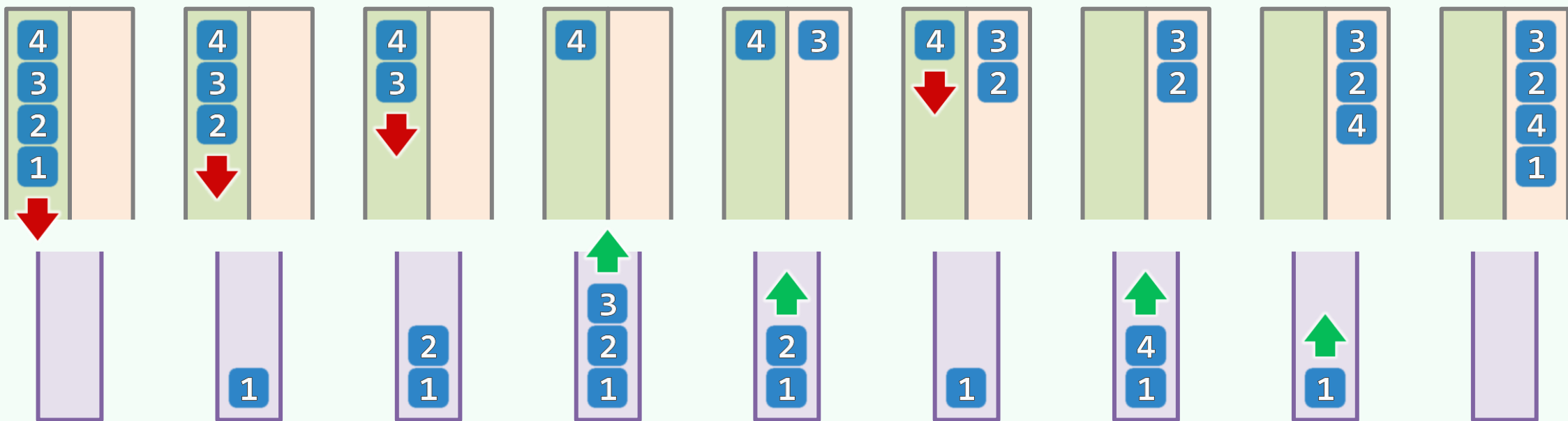
Stack Permutation



计数: $SP(n)$

❖ 同一输入序列, 可有多种栈混洗: $[1, 2, 3, 4 >$, $[4, 3, 2, 1 >$, $[3, 2, 4, 1 > \dots$

❖ 一般地, 对于长度为 n 的序列, 混洗总数 $SP(n) = ?$



❖ 栈混洗无非是排列, 且 $[3, 1, 2, \dots]$ 无法生成, 故有 $SP(n) < n!$; 更精准地呢?

计数: catalan(n)

$$\begin{aligned} \diamond SP(n) &= \sum_{k=1}^n SP(k-1) \cdot SP(n-k) \\ &= \text{catalan}(n) = (2n)! / (n+1)! / n! \end{aligned}$$

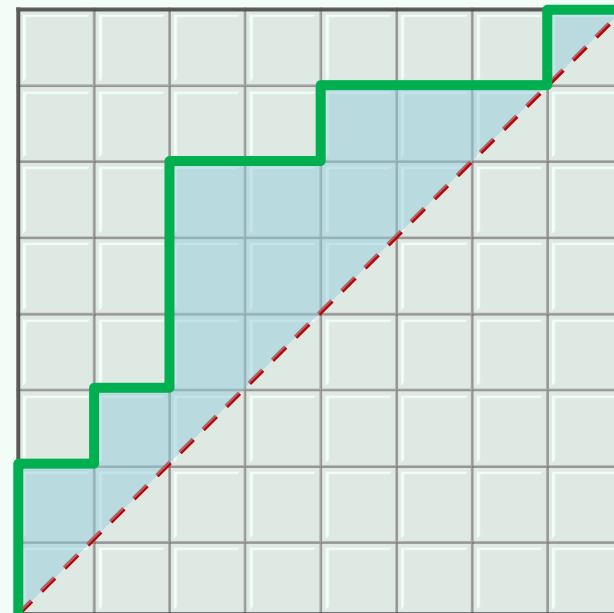
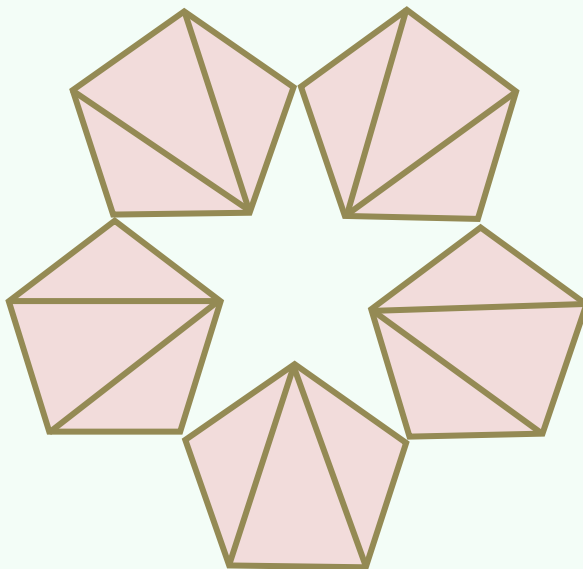
$$\diamond SP(1) = 1$$

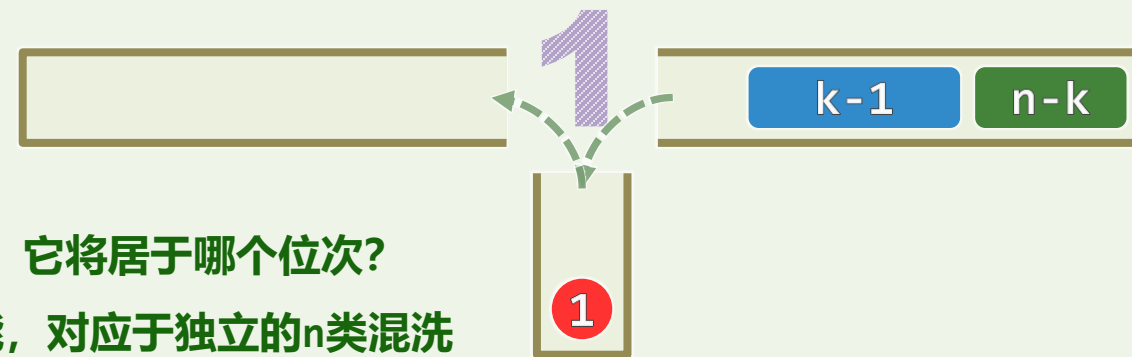
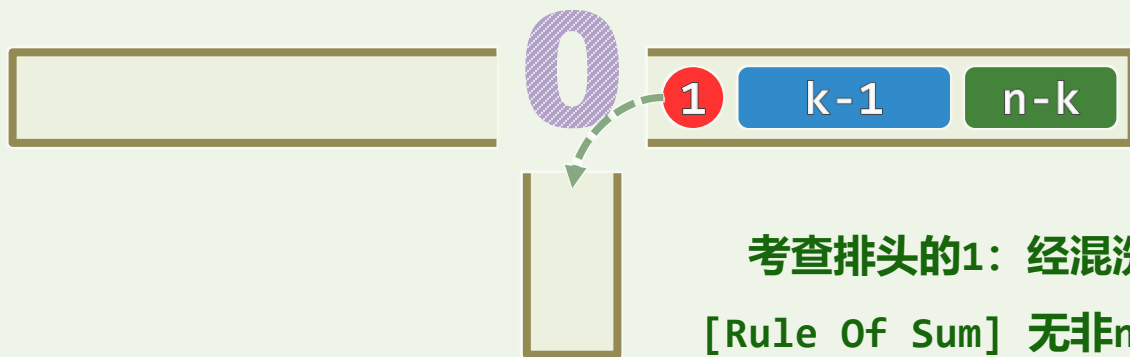
$$SP(2) = 4! / 3! / 2! = 2$$

$$SP(3) = 6! / 4! / 3! = 5$$

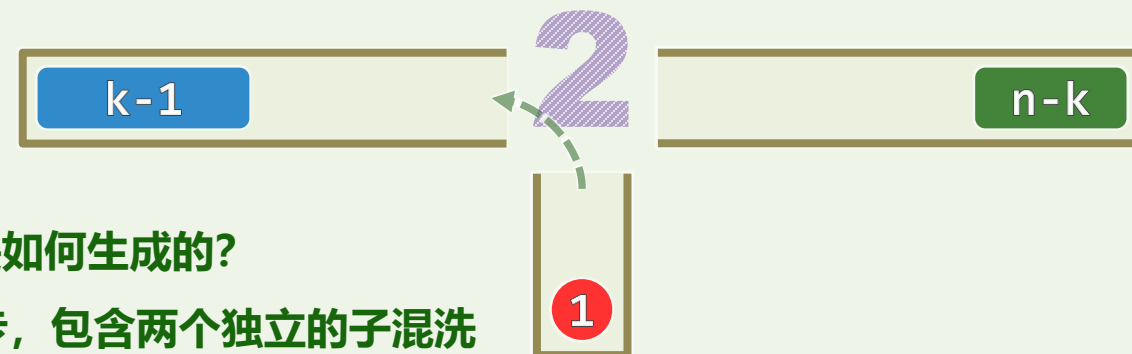
... ..

$$SP(6) = 12! / 7! / 6! = 132$$

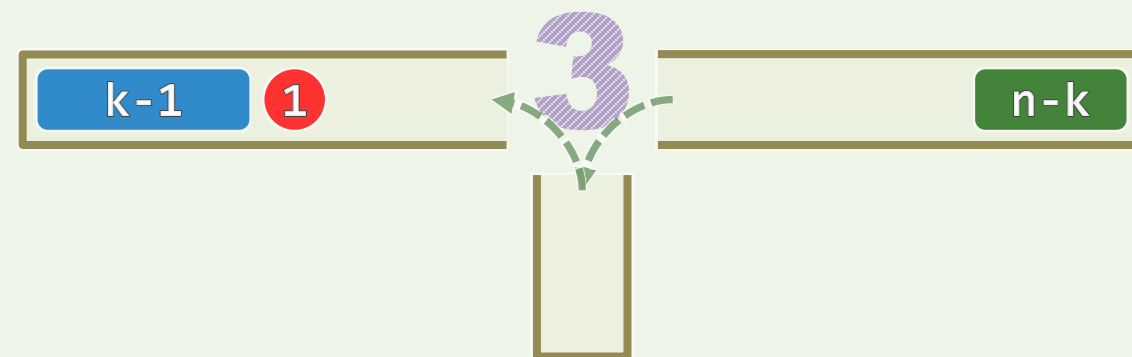
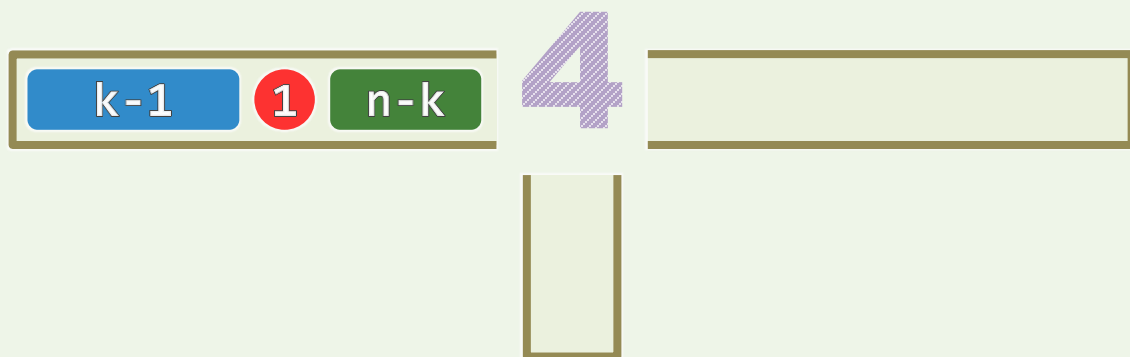




$$SP(n) = \sum_{k=1}^n SP(k-1) \cdot SP(n-k)$$



考查第k类：它们是如何生成的？
[Rule Of Product] 无非四步，包含两个独立的子混洗



甄别：检测禁形

❖ 输入序列 $\langle 1, 2, 3, \dots, n \rangle$ 的任一排列 $[p_1, p_2, p_3, \dots, p_n]$ 是否为栈混洗？

❖ $[3, 1, 2, \dots]$ 必然不是，因为...

按照以上加法原理，它属于第#2类：2应该在1之前，3应该在1之后

❖ $[\dots, 3, \dots, 1, \dots, 2, \dots]$ 也必然不是，因为...

既然3在1的前缀中，2也应该在

❖ **禁形**：对任何 $1 \leq i < j < k \leq n$ ， $[\dots, k, \dots, i, \dots, j, \dots]$ **必非**栈混洗，因为...

与上面同理，既然k在i的前缀中，j也应该在

❖ 反过来，不含**禁形**的序列，一定是栈混洗吗？

甄别：直接模拟

❖ 是的：一个序列是栈混洗，**当且仅当**其中不含任何禁形

如此，可得一个 $O(n^3)$ 的甄别算法

❖ 当然：对任何 $1 \leq i < j \leq n - 1$ ，必非栈混洗 $[\dots, j + 1, \dots, i, \dots, j, \dots, \rangle$

反过来，不含此类**禁形**的序列，也必然是栈混洗

如此，可得一个 $O(n^2)$ 的甄别算法

❖ 返璞归真，借助栈直接模拟，只需 $O(n)$ 时间

逐个检视各目标元素：

- 若在（或能够腾挪至）S顶部，则执行B操作
- 否则（在S中，却非栈顶），则无法完成混洗

甄别：算法

```
❖ Rank spCheck( Vector<Rank>& B ) { //B = unsort[1,n]

    Stack<Rank> S; Rank A = 1; //中转栈、输入栈, 后者<1,2,3,...,n]只需记录栈顶

    for ( Rank n = B.size(), r = 0; r < n; S.pop(), r++ ) //模拟: 逐项尝试B操作

        if ( S.empty() || S.top() != B[r] )

            if ( n < A || B[r] < A ) //A已空, 或者不含B[r]

                return r; //卡住

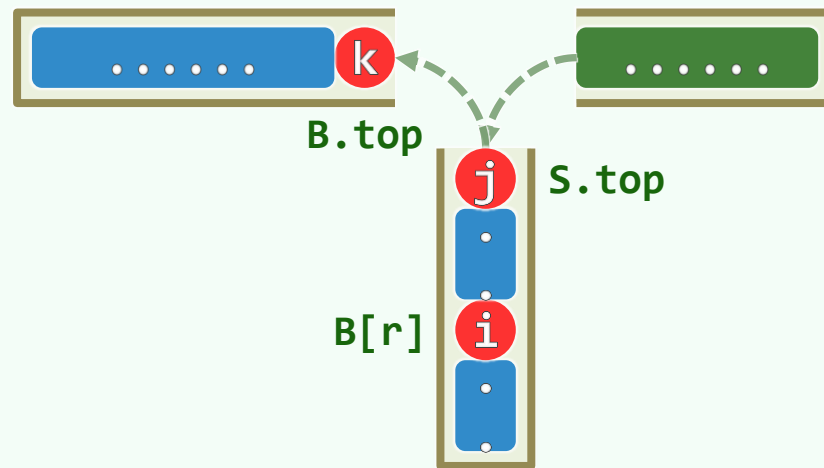
            else

                do S.push( A++ ); //A操作

                while ( S.top() != B[r] );

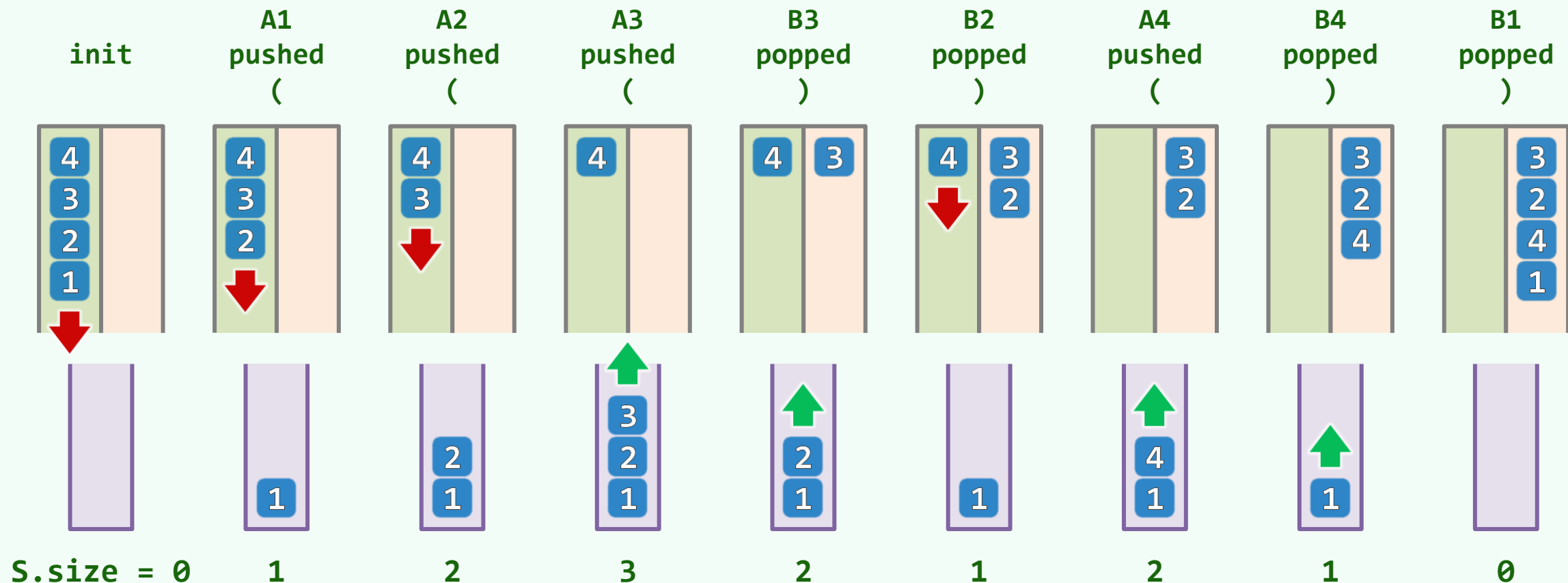
    return B.size(); //成功

} //spCheck, O(n)
```



括号匹配

❖ 每一栈混洗，都对应于n次A操作与n次B操作构成的某一序列；且在任一前缀中，A操作不少于B操作



❖ 将A、B操作对应于左、右括号：n个元素的栈混洗，——对应于n对括号的匹配表达式

04-G

栈与队列

队列接口与实现

邓俊辉

deng@tsinghua.edu.cn

也有個先來後到！甚麼官人的伴當要換座頭！老爺不換！

操作与接口

❖ 队列 (queue) 也是受限的序列

❖ 只能在队尾插入 (查询) :

- enqueue() / rear()

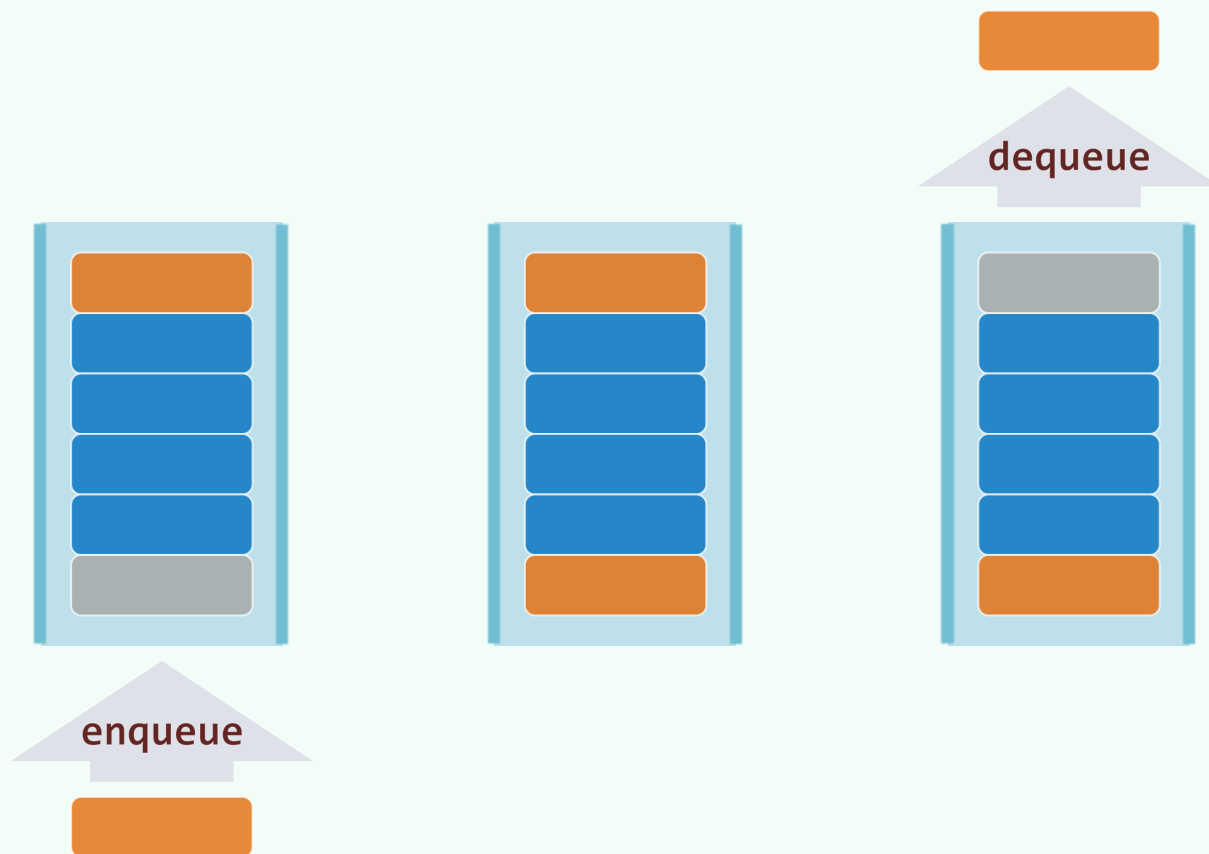
❖ 只能在队头删除 (查询) :

- dequeue() / front()

❖ 先进先出 (FIFO)

后进后出 (LILO)

❖ 扩展接口: getMax()...



实例

操作	>-- 队列 -->				输出
Queue()					
empty()					true
enqueue(E)	E				
enqueue(C)	C	E			
dequeue()	C				E
enqueue(G)	G	C			
enqueue(C)	C	G	C		
front()	C	G	C		C
empty()	C	G	C		false

操作	>-- 队列 -->						输出
enqueue(K)	K	C	G	C			
size()	K	C	G	C			4
enqueue(F)	F	K	C	G	C		
empty()	F	K	C	G	C		false
enqueue(G)	G	F	K	C	G	C	
dequeue()	G	F	K	C	G		C
dequeue()	G	F	K	C			G
front()	G	F	K	C			C
size()	G	F	K	C			4

实现：既然属于**线性序列**，故可直接基于向量或列表**派生**

```
template <typename T> struct Queue: public List<T> { //原有接口一概沿用
```

```
void enqueue( const T & e ) { insertLast( e ); } //入队
```

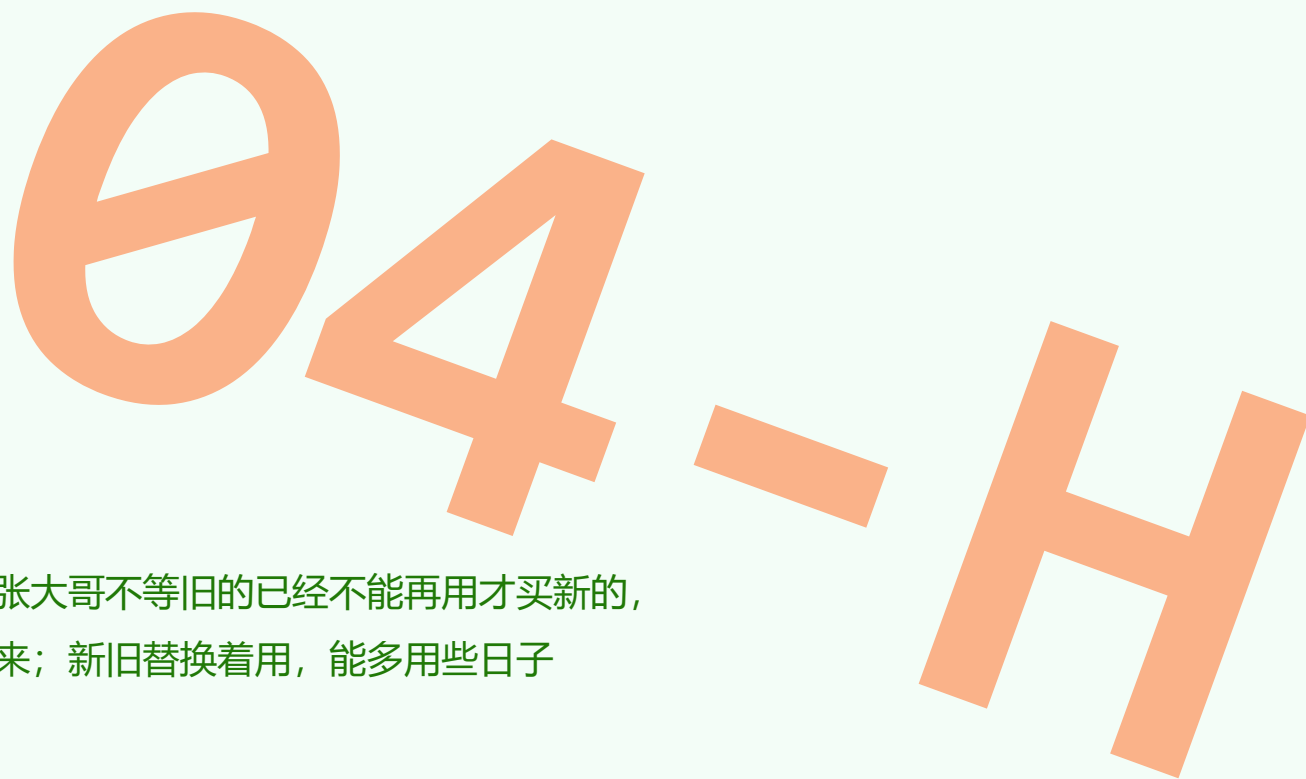
```
T dequeue() { return remove( first() ); } //出队
```

```
T & front() { return first()->data; } //队首
```

```
}; //以列表首/末端为队列头/尾——颠倒过来呢？
```

❖ 确认：如此实现的队列接口，均只需 $\mathcal{O}(1)$ 时间

❖ 课后：基于**向量**，派生出队列模板类——如此实现的各接口，效率如何？



栈与队列

队列应用

墙上一溜挂着五个烟斗。张大哥不等旧的已经不能再用才买新的，而是使到半路就买个新的来；新旧替换着用，能多用些日子

這三個圍住呂布，轉燈兒般廝殺。八路人馬，都看得呆了

邓俊辉

deng@tsinghua.edu.cn

轮转调度

❖ 一组客户 (client) 共享同一资源时, 如何兼顾**公平与效率**?

比如, 多个应用程序共享CPU, 实验室成员共享打印机, ...

❖ RoundRobin //轮转调度

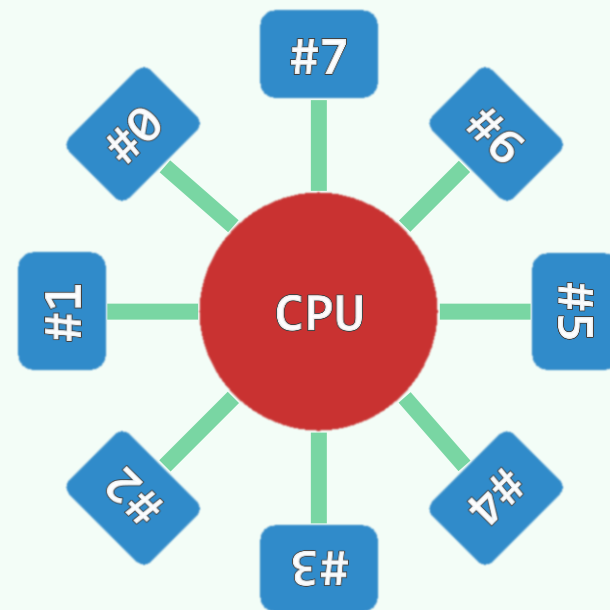
```
Queue Q( clients ); //共享资源的所有客户组成队列
```

```
while ( ! ServiceClosed() ) //在服务关闭之前, 反复地
```

```
    e = Q.dequeue(); //令队首的客户出队, 并
```

```
    serve( e, quantum ); //接受时长为quantum的服务, 然后
```

```
    Q.enqueue( e ); //重新入队
```



前缀和

❖ 任给序列: $A = \{a_k \mid 0 \leq k < n\}$

计算序列: $X = \{x_k = \sum_{i=0}^k a_i \mid 0 \leq k < n\}$

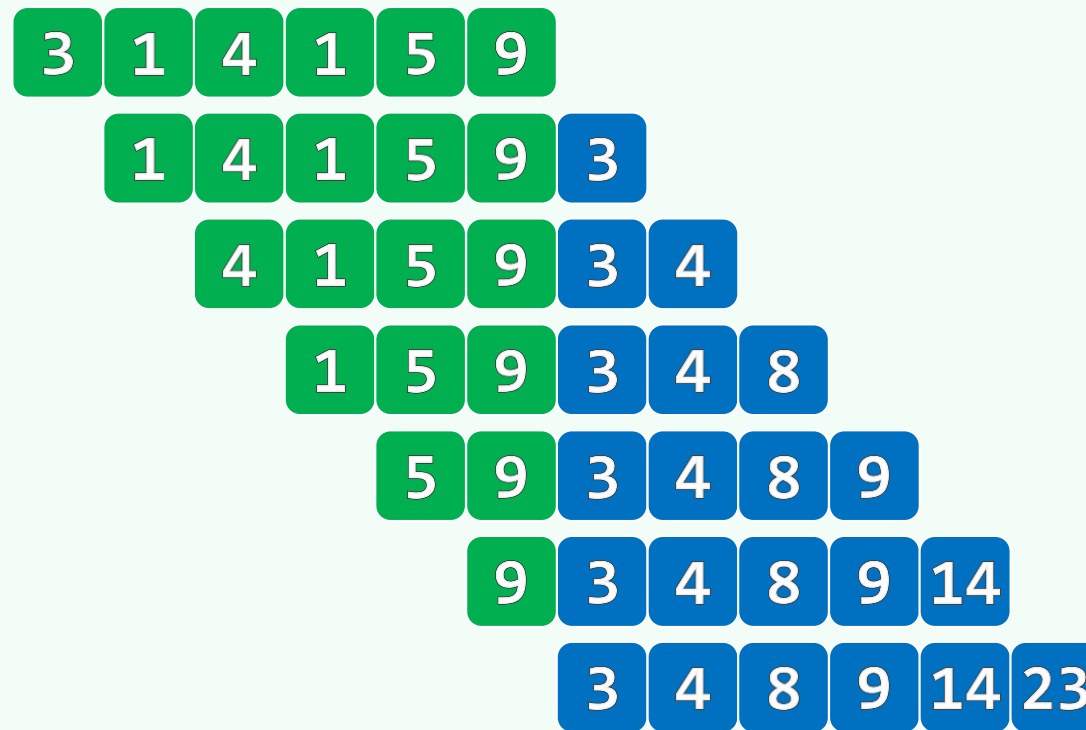
❖ void qPreSum(Queue<int>& A, Rank n) {

 int sum = 0; //前缀和

 while (n--) //逐项累加

 A.enqueue(sum += A.dequeue());

} //qSum, $O(n)$



❖ 进一步地, 计算 $S(w) = \{s_k = \sum_{i=k}^{k+w-1} a_i \mid 0 \leq k \leq n - w\}$

$= \{s_k = x_{k+w-1} - x_{k-1} \mid 0 \leq k \leq n - w\}$, 补充 $x_{-1} = 0$

滑动窗口和

```
❖ void qWinSum( Queue<int>& A, Rank n, Rank w ) { //w<=n
```

qPreSum(A, n); //先求前綴和

```
Queue<int> W; W.enqueue( 0 ); //建立窗口
```

```
for ( Rank i = 1; i < w; i++ ) //充满窗口
```

```
W.enqueue( A.dequeue() );
```

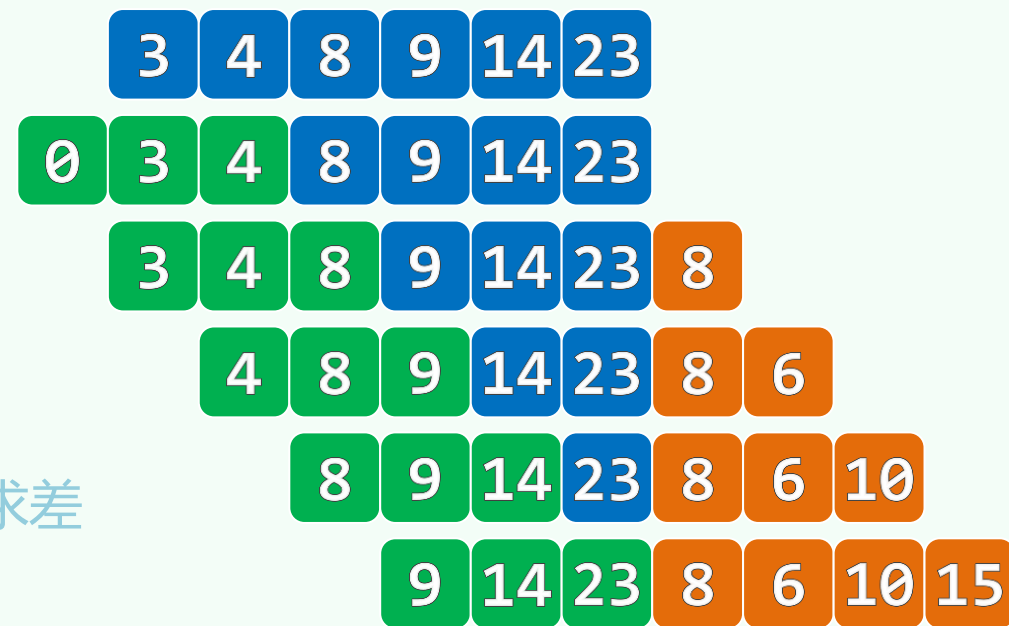
```
for ( Rank i = w-1; i < n; i++ ) { //逐项
```

```
A.enqueue( A.front() - W.dequeue() ); //求差
```

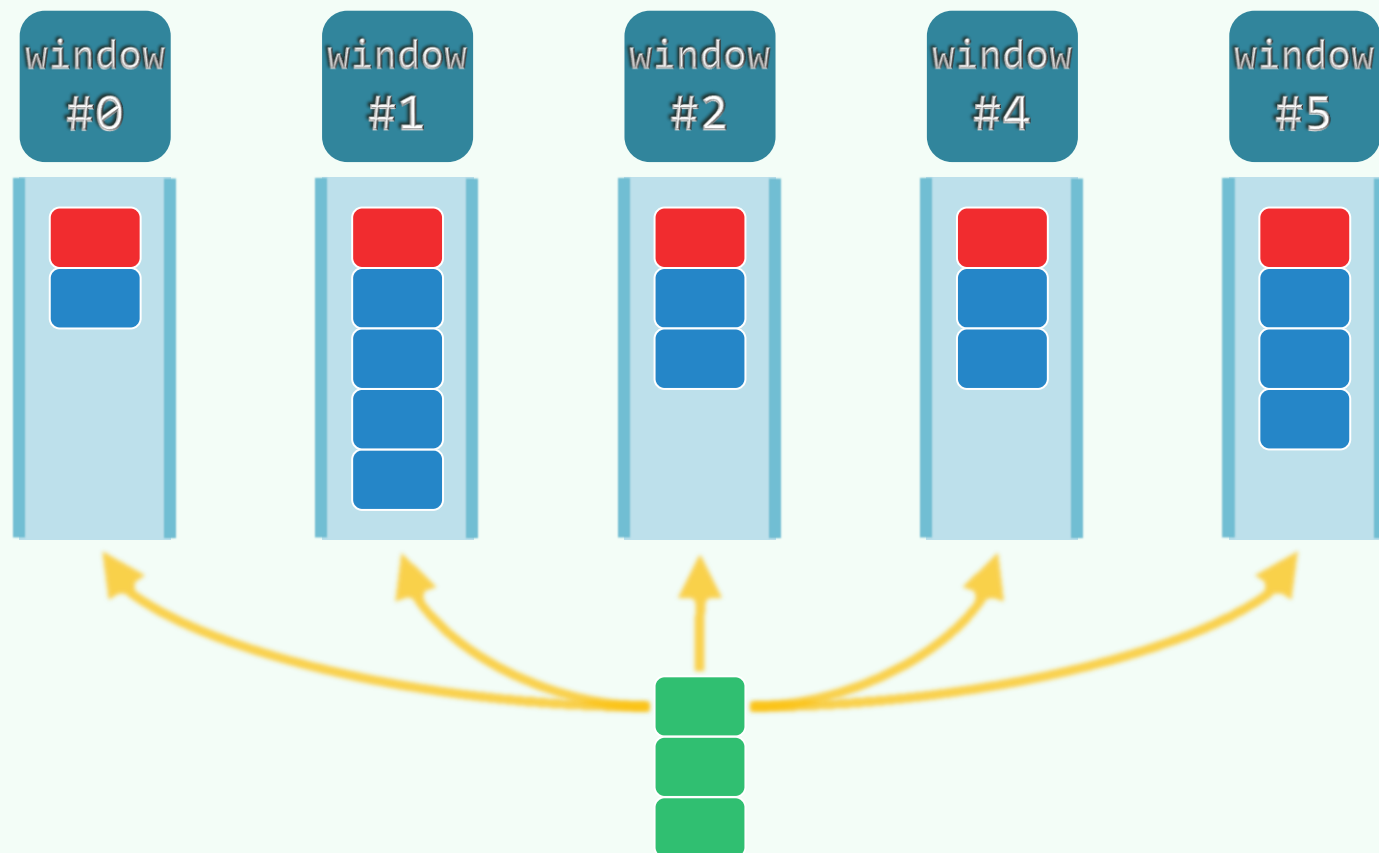
```
W.enqueue( A.dequeue() ); //滑动窗口
```

```
} //for
```

```
} //qWinSum,  $O(n)$ 
```



银行服务模拟：模型



银行服务模拟：算法

```
void simu( int winds, int duration ) { //窗口数、营业时长

    Queue<int>* wind = new Queue<int>[wins]; //窗口各维护一个队列，记录顾客们的服务时长

    while ( duration-- ) { //在下班之前，每隔一个单位时间

        for ( int i = 0; i < winds; i++ ) //各非空队列的队首顾客接受

            if ( !wind[i].empty() && ( 1 > wind[i].front()-- ) //单位时长的服务

                wind[i].dequeue(); //服务完毕则出列，后继者接替

        if ( rand() % ( 1 + winds ) ) //新顾客以wins/(wins + 1)的概率到达

            wind[ rand()%winds ].enqueue( 1 + rand()%99 ); //随机选窗口、定时长

    } //for

    delete [] window; //释放所有队列

} //simu
```

04-I

栈与队列

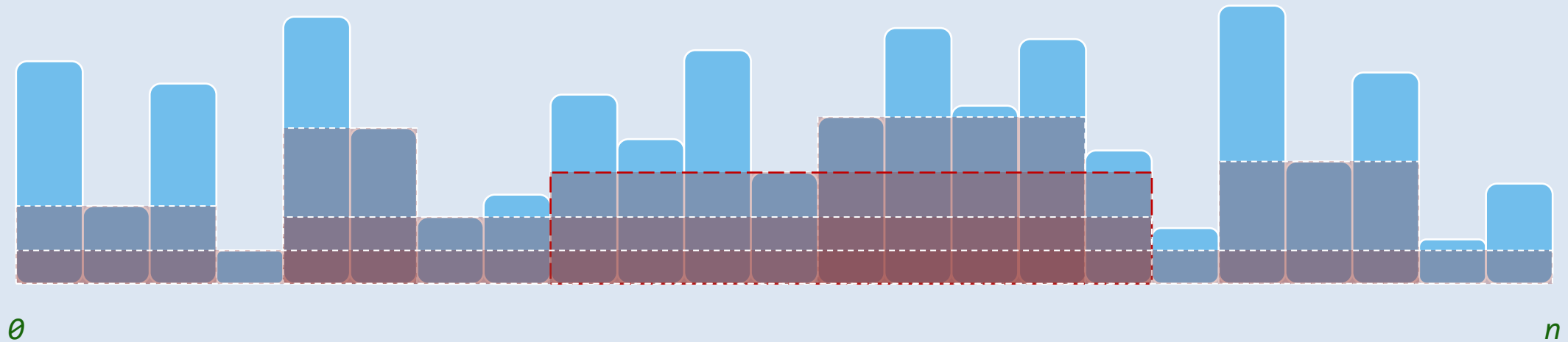
直方图内最大矩形

就这么着，我有了一所严丝密缝、涂抹灰泥的木板房子，七英尺宽，十五英尺长，立柱有八英尺高...

邓俊辉

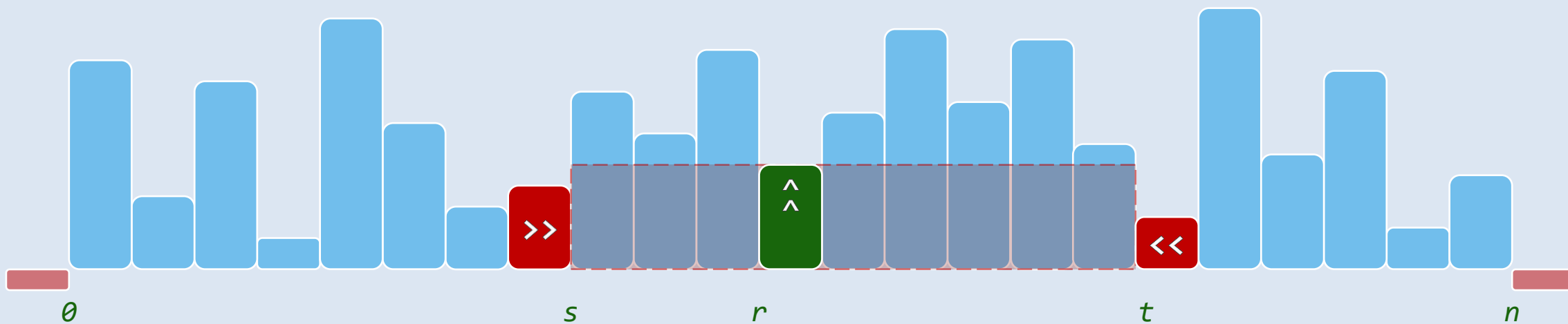
deng@tsinghua.edu.cn

Maximum Rectangle



- ❖ Let $H[0,n)$ be a histogram of non-negative integers
- ❖ How to find the largest orthogonal rectangle in $H[]$?
- ❖ To eliminate possible ambiguity
we can, for example, choose the **LEFTMOST** one

Maximal Rectangles



❖ Maximal rectangle supported by $H[r]$: $maxRect(r) = H[r] \cdot (t(r) - s(r))$

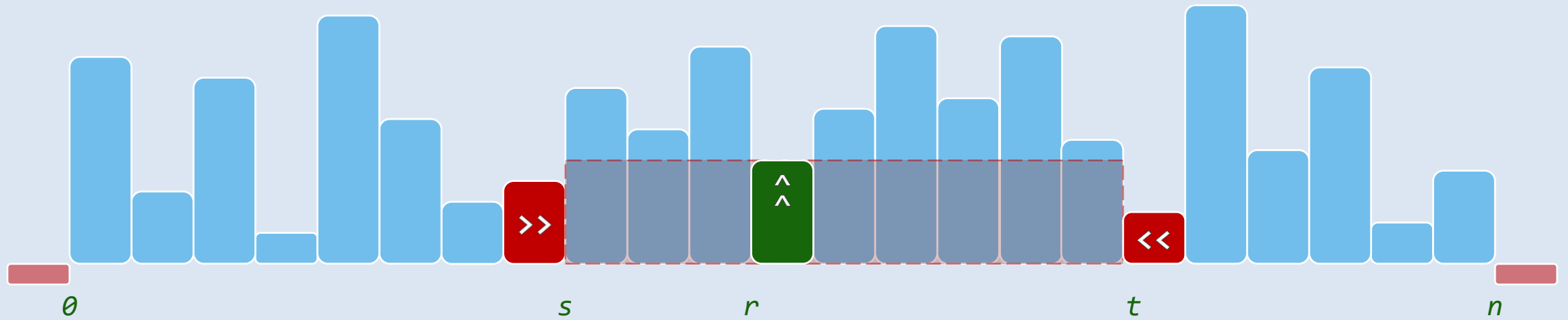
$$s(r) = \max\{ k \mid 0 \leq k \leq r \text{ and } H[k-1] < H[r] \}$$

where

$$t(r) = \min\{ k \mid r < k \leq n \text{ and } H[r] > H[k] \}$$

❖ The maximum rectangle must be a maximal one

Brute-force



❖ Determining $s(r)$ and $t(r)$ for all r 's requires $\mathcal{O}(n^2)$ time

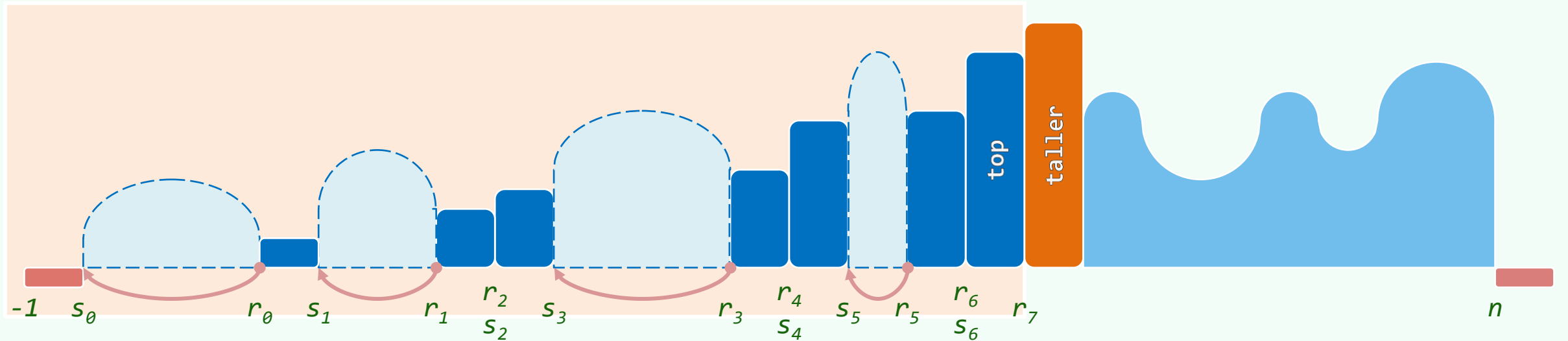
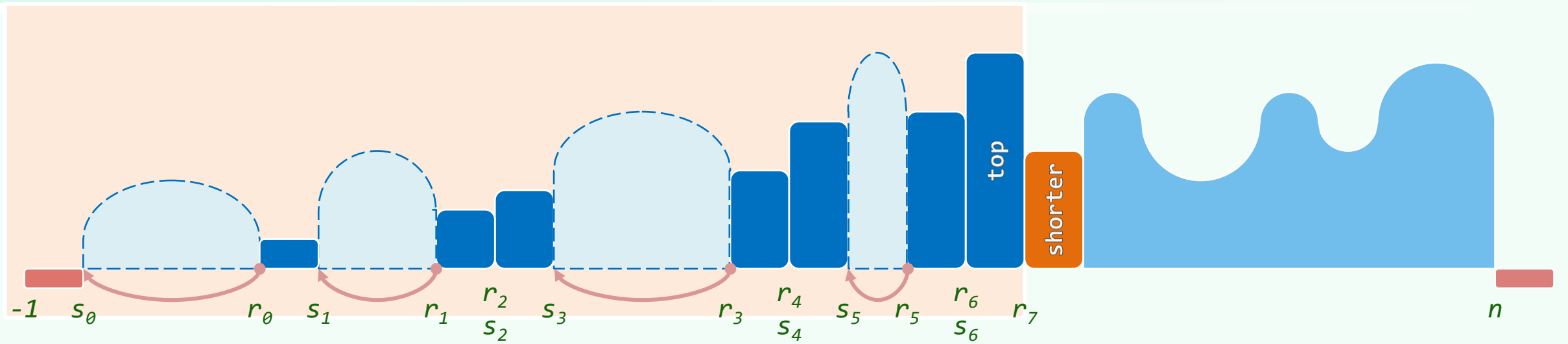
$$s(r) = \max\{ k \mid 0 \leq k \leq r \text{ and } H[k-1] < H[r] \}$$

where

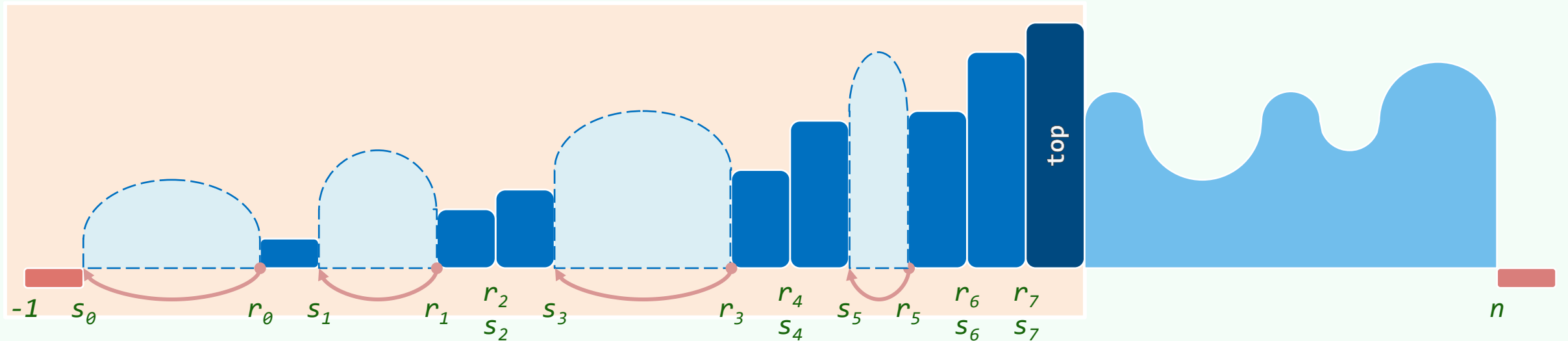
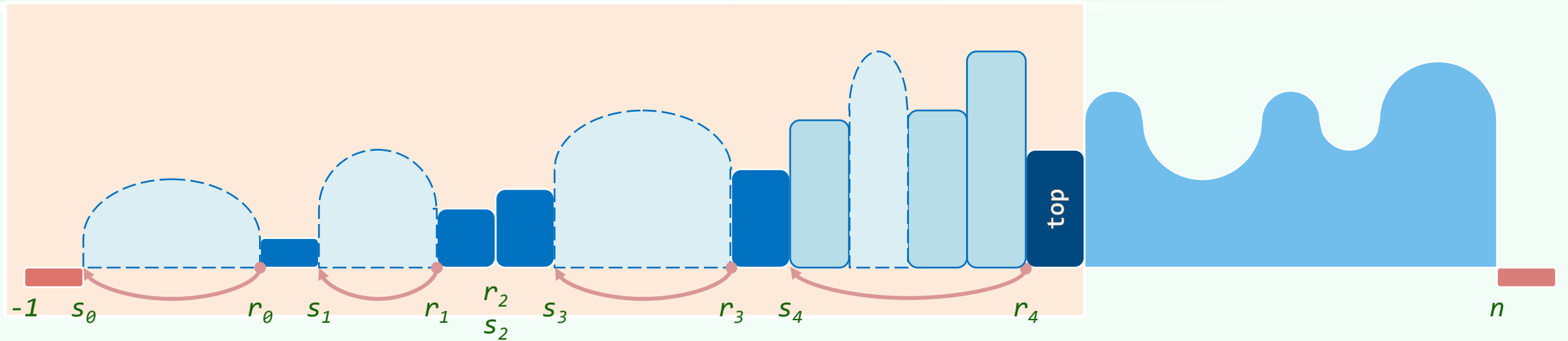
$$t(r) = \min\{ k \mid r < k \leq n \text{ and } H[r] > H[k] \}$$

❖ Actually, all $s(r)$'s can be determined by a **LINEAR** scan of the histogram ...

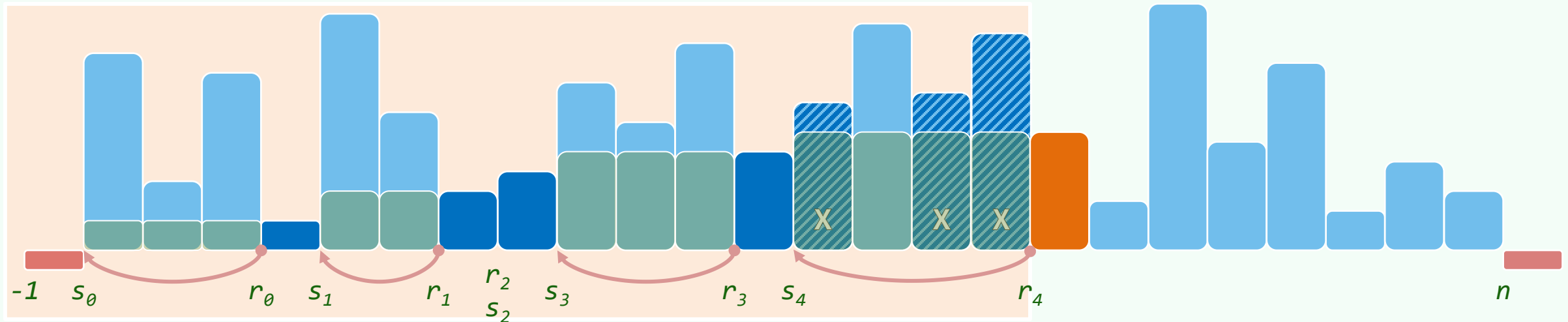
Using Monotonic Stack: 2 Cases: Pre



Using Monotonic Stack: 2 Cases: Post



Using Monotonic Stack: Algorithm



```
Rank* s = new Rank[n]; Stack<Rank> S;
```

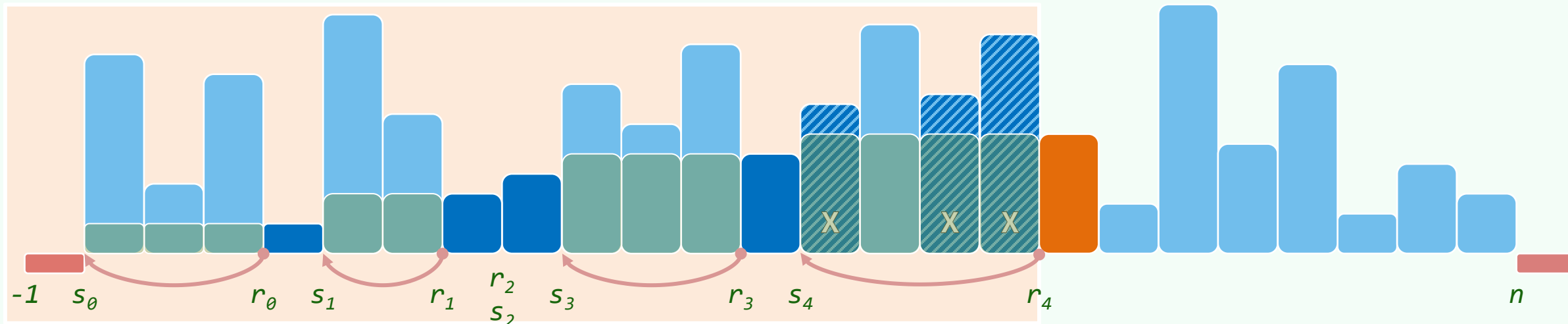
```
for ( Rank r = 0; r < n; r++ ) //try using SENTINEL for simplicity by yourself
```

```
    while ( !S.empty() && ( H[S.top()] >= H[r] ) ) S.pop(); //until H[top] < H[r]
```

```
    s[r] = S.empty() ? 0 : 1 + S.top();    S.push(r); //S is always ASCENDING
```

```
while( !S.empty() ) S.pop();
```

Using Monotonic Stack: Loop Invariant & Correctness



❖ All bars are scanned and pushed into S **in turn**

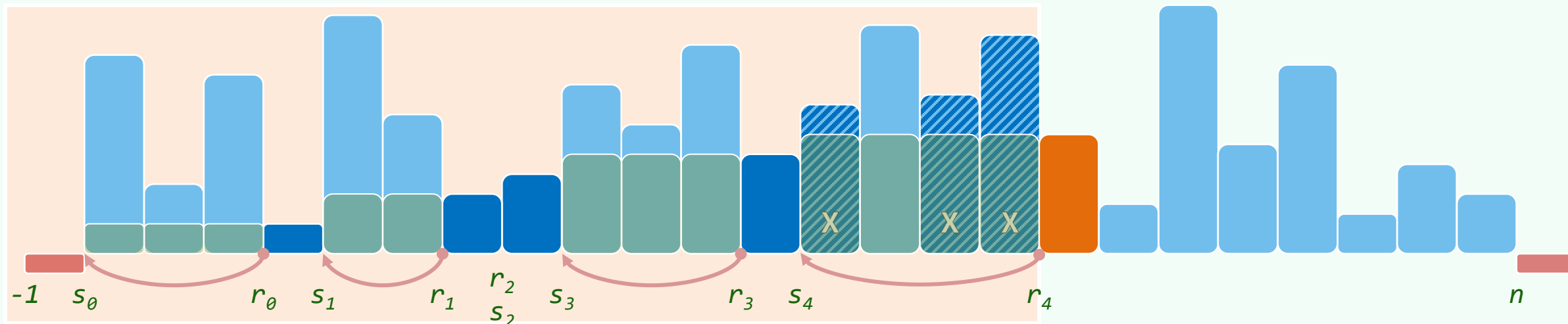
❖ After each iteration of the outer loop, S stores a **chain** in terms of $s[]$

$$S[S.size() - 1] = S.top() = r \quad \text{and} \quad \forall 0 \leq k < S.size(), \quad S[k-1] = s[S[k]] - 1$$

❖ Every bar $H[k]$ is popped when it will be of **no use** thereafter, i.e.

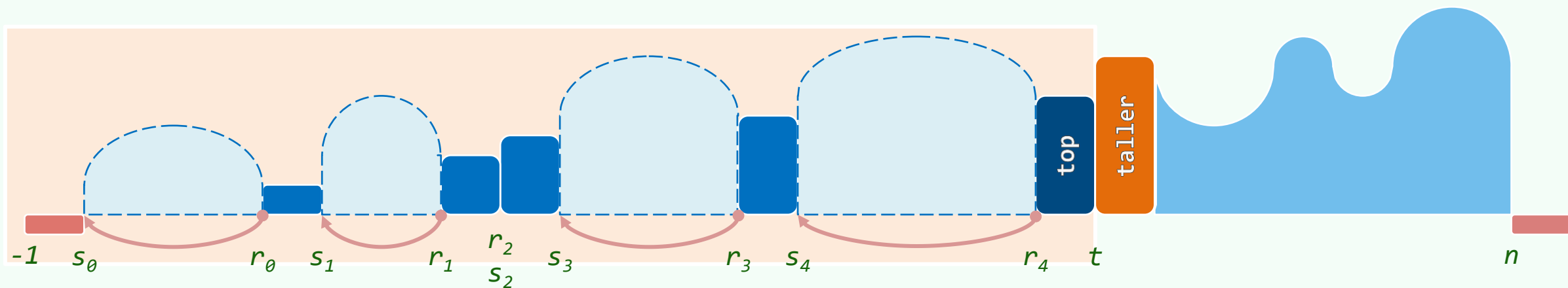
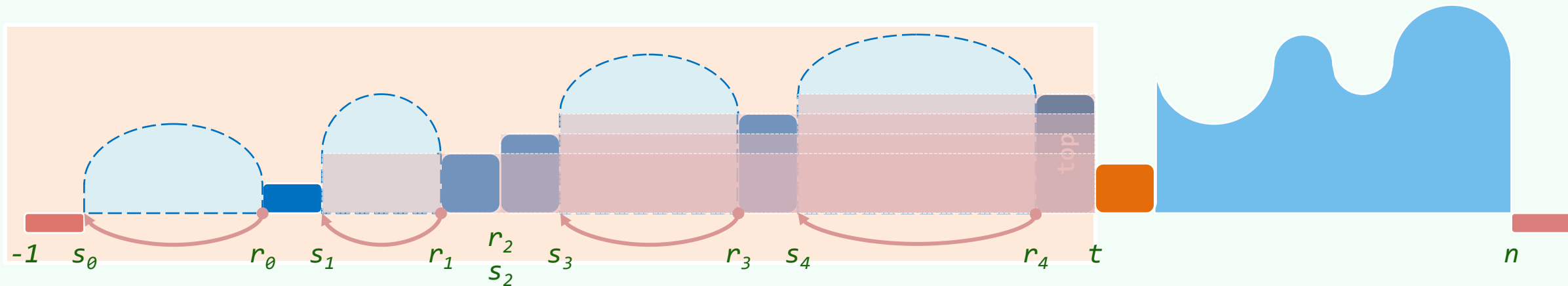
$$\forall 0 \leq k < r \text{ but } k \notin S, \quad \nexists r \leq t \text{ s.t. } k+1 = s(t)$$

Using Monotonic Stack: Complexity

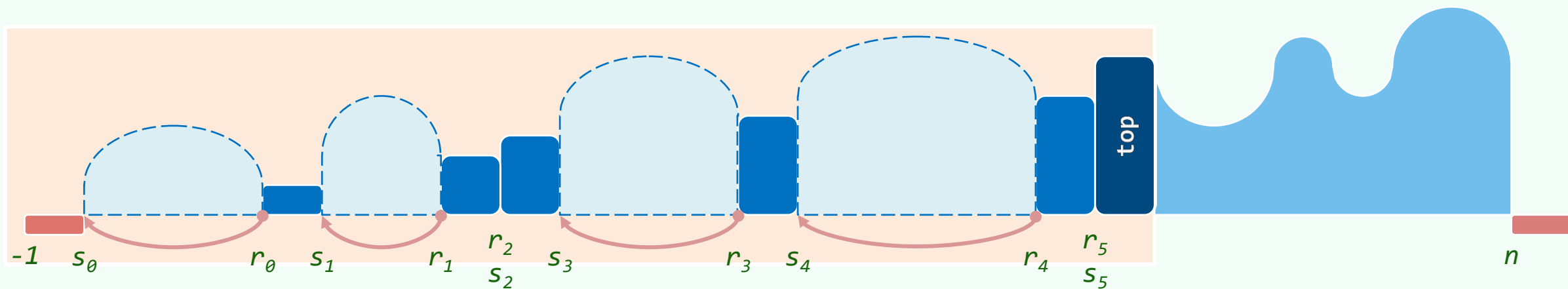
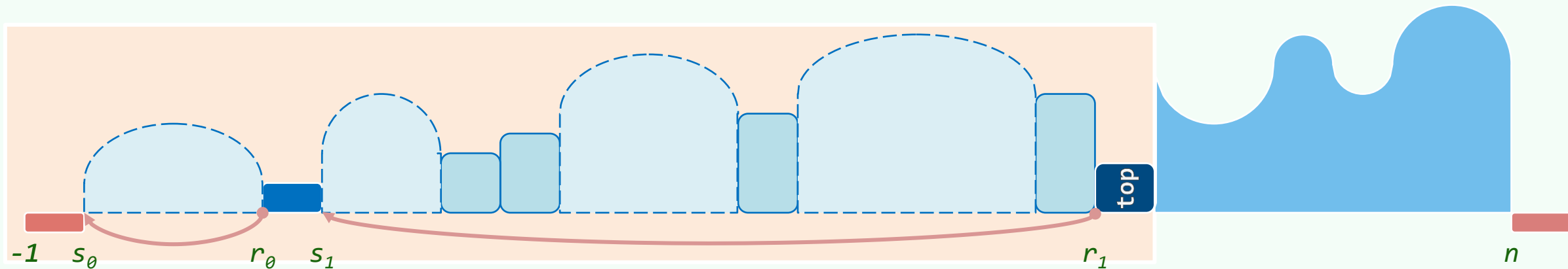


- ❖ And $t(r)$'s can be determined by another scan in the **REVERSED** direction
- ❖ Hence all maximal rectangles can be computed in $\mathcal{O}(n)$ time and using $\mathcal{O}(n)$ space
- ❖ However, what if the histogram is given in an **IN-PLACE** and **ON-LINE** manner?
Note that, the $t(r)$'s **CAN'T** be determined until the **ENTIRE** input is ready
- ❖ Is it possible to compute **BOTH** $s(r)$'s and $t(r)$'s by a **SINGLE** scan? [//on-fly](#)

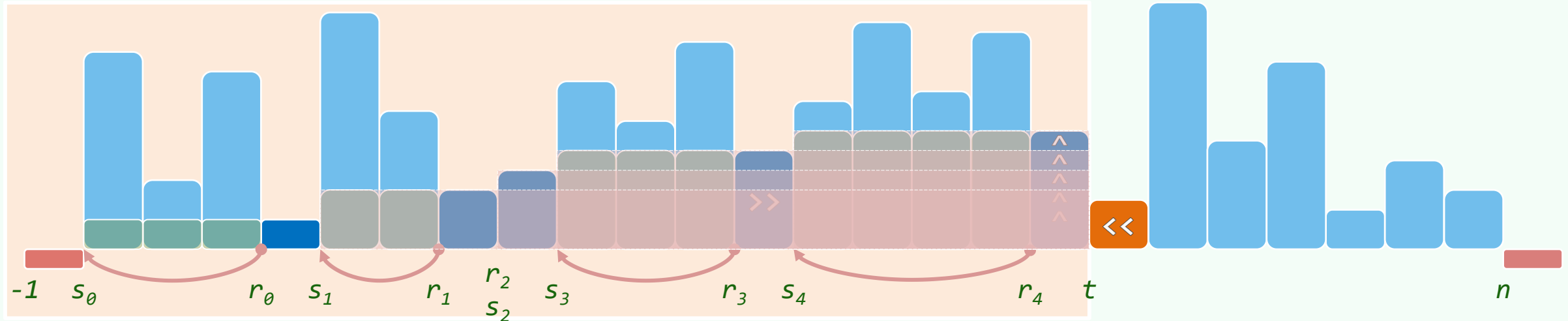
One-Pass Scan: 2 Cases: Pre



One-Pass Scan: 2 Cases: Post

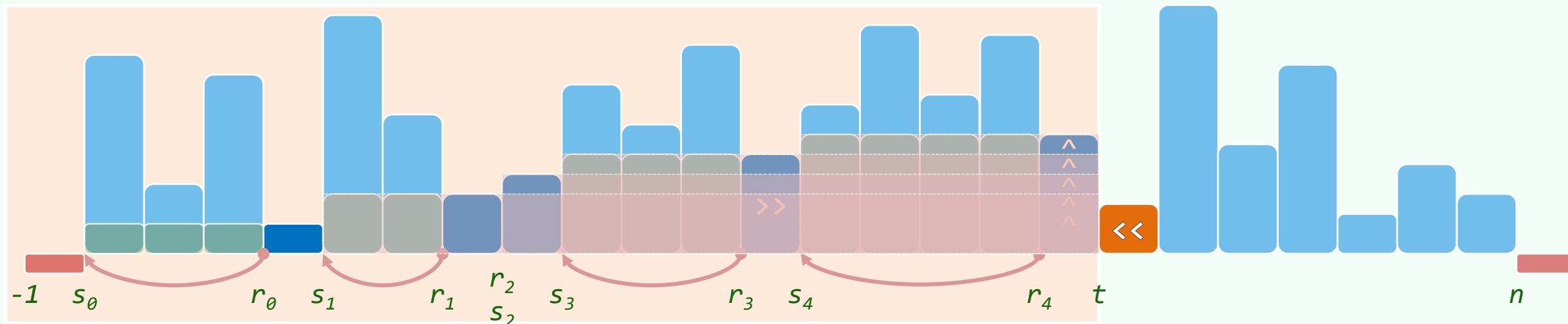


One-Pass Scan: Algorithm



```
Stack<Rank> SR;  uint64_t maxRect = 0; //SR.2ndTop() == s(r)-1 & SR.top() == r
for ( Rank t = 0; t <= n; t++ ) //amortized- $\mathcal{O}(n)$ 
    while ( !SR.empty() && ( t == n || H[SR.top()] >= H[t] ) )
        Rank r = SR.pop(), s = SR.empty() ? 0 : SR.top() + 1;
        maxRect = max( maxRect, H[r] * ( t - s ) );
    if ( t < n ) SR.push( t );
return maxRect;
```

One-Pass Scan: Loop Invariant & Correctness



❖ Again, **at** each iteration of the **outer** loop, we always have

$$\forall 0 \leq k < SR.size(), \quad SR[k-1] = s[SR[k]] - 1$$

❖ For each bar r **popped** in the **inner** loop, we have

$$t(r) = t \quad \text{and} \quad s[r] = SR.top() - 1$$

04.5

栈与队列

最大栈 + 最大队列

見賢思齊焉，見不賢自省也

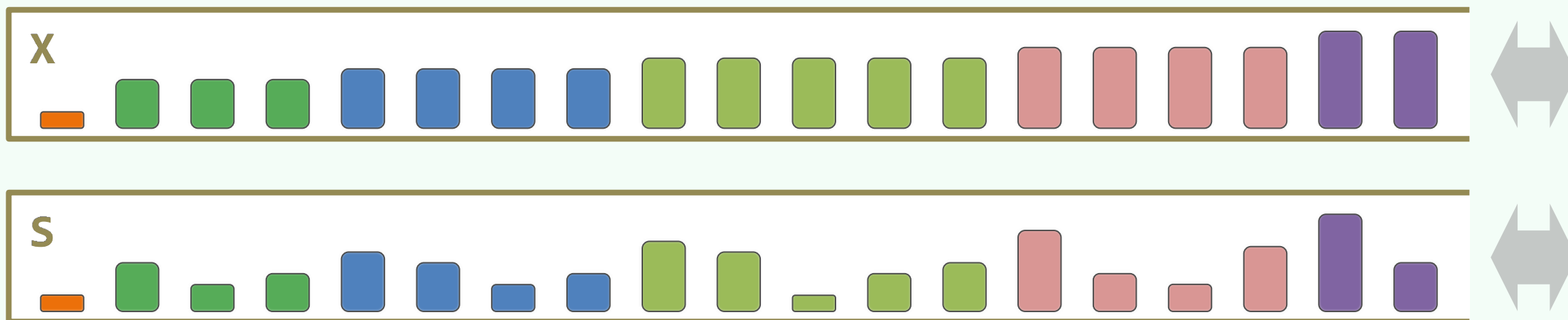
A hero is born among a hundred, a wise man is found among a thousand, but an accomplished one might not be found even among a hundred thousand men.

邓俊辉

deng@tsinghua.edu.cn

$$\text{Max Stack} = \text{push} + \text{pop} + \text{getMax} = S + X$$

❖ X: 每个元素, 都是S中对应**后缀里的最大者**; 自底向顶, 顺次递增; 栈顶即是最大



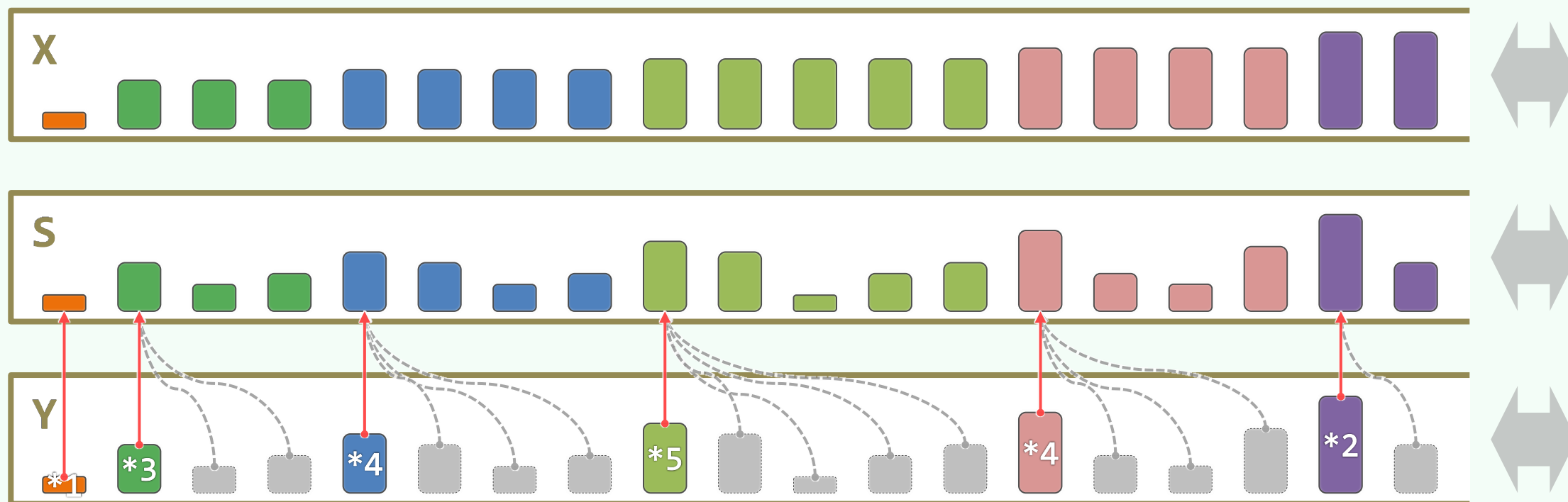
❖ `getMax()` { return X.**top**(); } *//O(1)*

❖ `pop()` { X.pop(); return S.pop(); } *//O(1)*

❖ `push( e )` { X.push(**max**(X.top(), e)); S.push(e); } *//O(1)*

Max Stack: 压缩空间

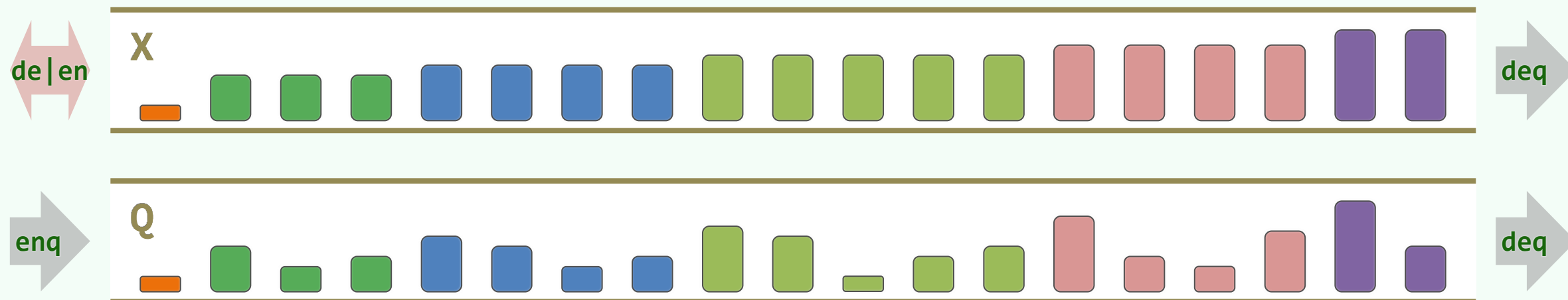
❖ X去重, 升级至N: S中的**关键**元素, 才会记录在Y中; 只需记录对应的**引用**, 外加一个**计数器**



❖ `S.push()`和`S.pop()`, 分别对应于计数器的**增|生**、**减|灭**

Max Queue = enqueue + dequeue + getMax = Q + X

❖ dequeue() { X.dequeue(); return Q.dequeue(); } //O(1)



❖ enqueue(e) //最坏情况O(n), 且可能持续发生

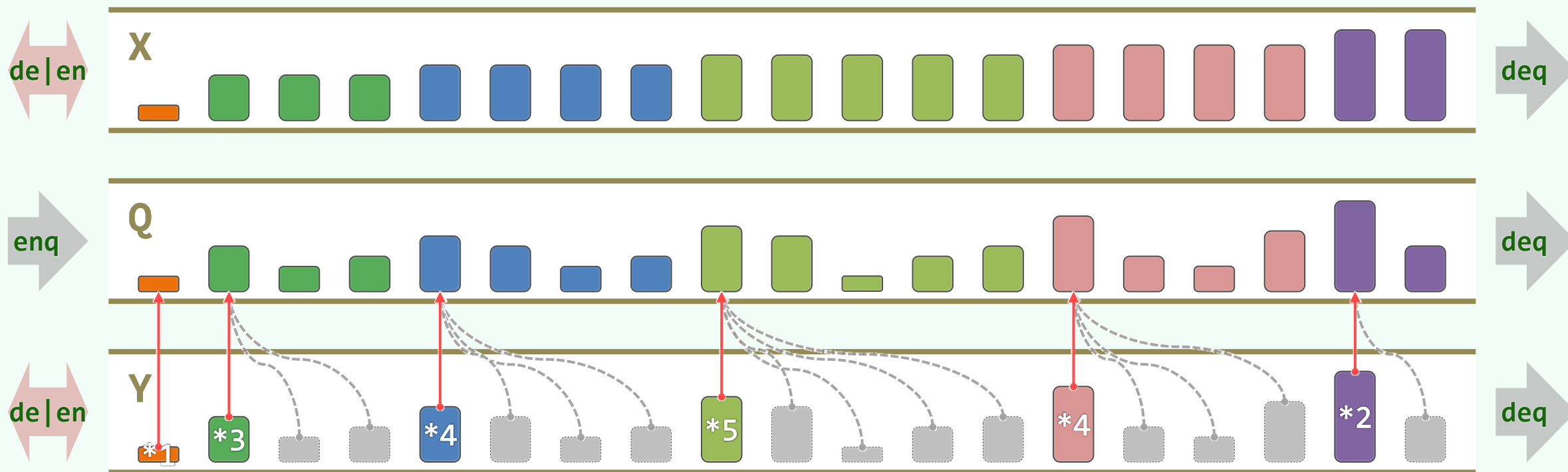
for (Q.enqueue(e), x = X.rear(); x && (e > x->data); x = x->pred) //见贤

x->data = e; //思齐

X.enqueue(e);

Max Queue: 压缩空间

❖ X去重，**升级**至Y: Q中的**关键**元素，才会记录在Y中...



❖ **Q.enqueue()**和**Q.dequeue()**，分别对应于计数器的**合|生**、**减|灭** ~ **均摊**...

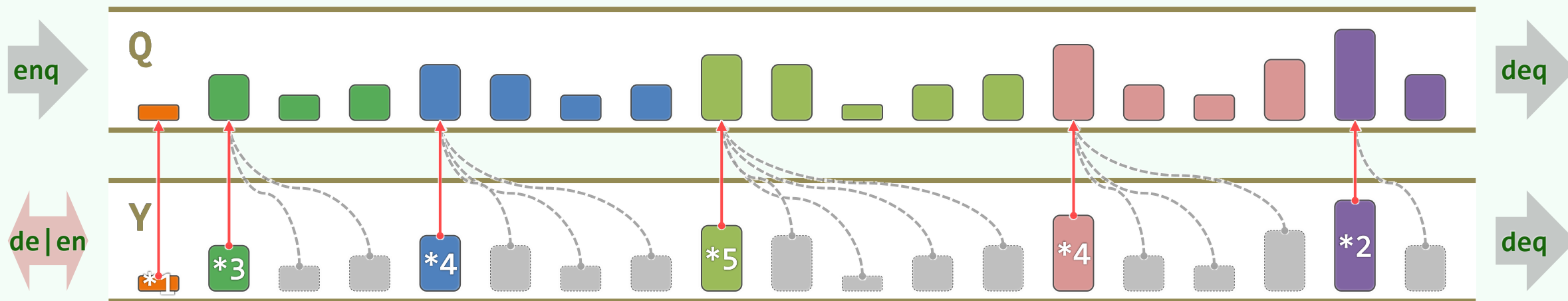
Max Queue: 均摊成本

❖ enqueue(e) //均摊 $O(1)$

```
Q.enqueue(e); c = 1;
```

```
while ( !Y.empty() && (e ==> Y.rear()->data) ) c += Y.dequeueRear()->count;
```

```
Y.enqueue( <&e, c> );
```



❖ dequeue() { if (--Y.top().count < 1) Y.dequeue(); return Q.dequeue(); } //均摊 $O(1)$

04-k

栈与队列

双栈当队

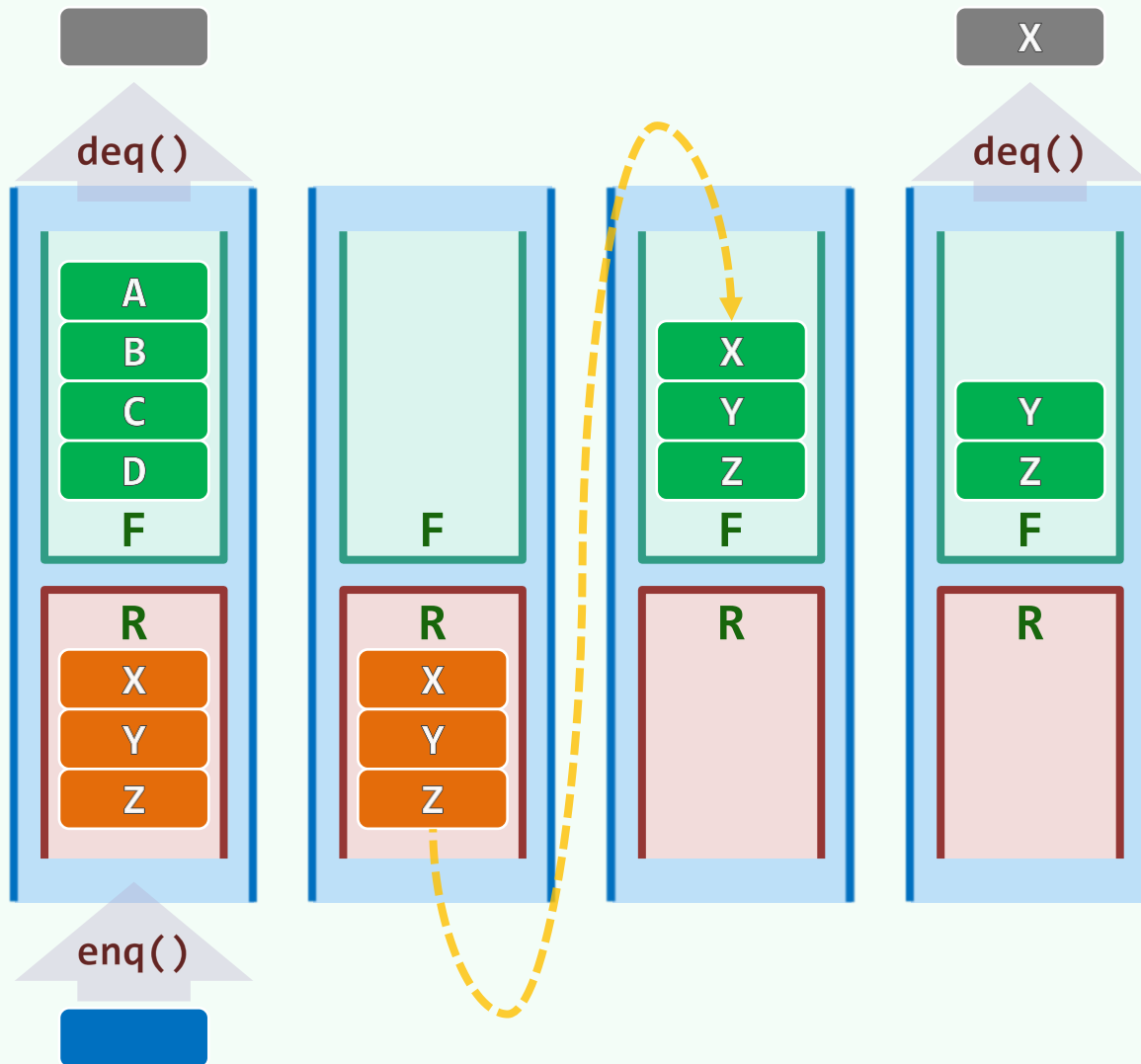
我们也不用当场付款，要了什么东西都由店家记在一个小账本上，
每两星期结一次账

不恒其德，或承之羞，贞吝

邓俊辉

deng@tsinghua.edu.cn

Queue = Stack x 2



❖ `def Q.enqueue(e)`

`R.push(e);`

❖ `def Q.dequeue() # 0 < Q.size()`

`if ( F.empty() )`

`while ( !R.empty() )`

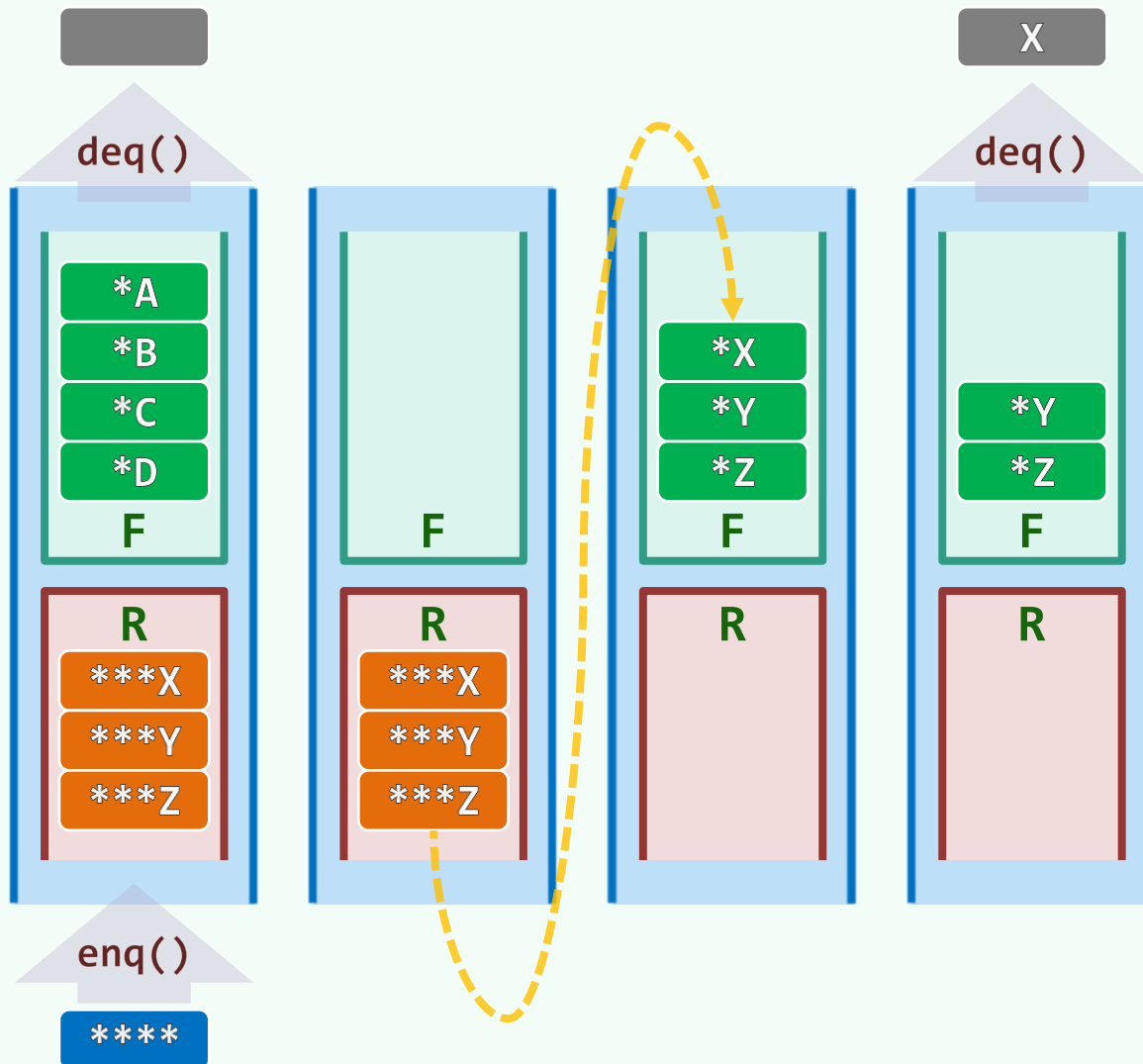
`F.push( R.pop() );`

`return F.pop();`

❖ Best/worst case: $O(1)/O(n)$

Average? Amortization!

Accounting | 记账法 | 归因法



❖ Assign **4** coins to each element

//deposit | pre-paid

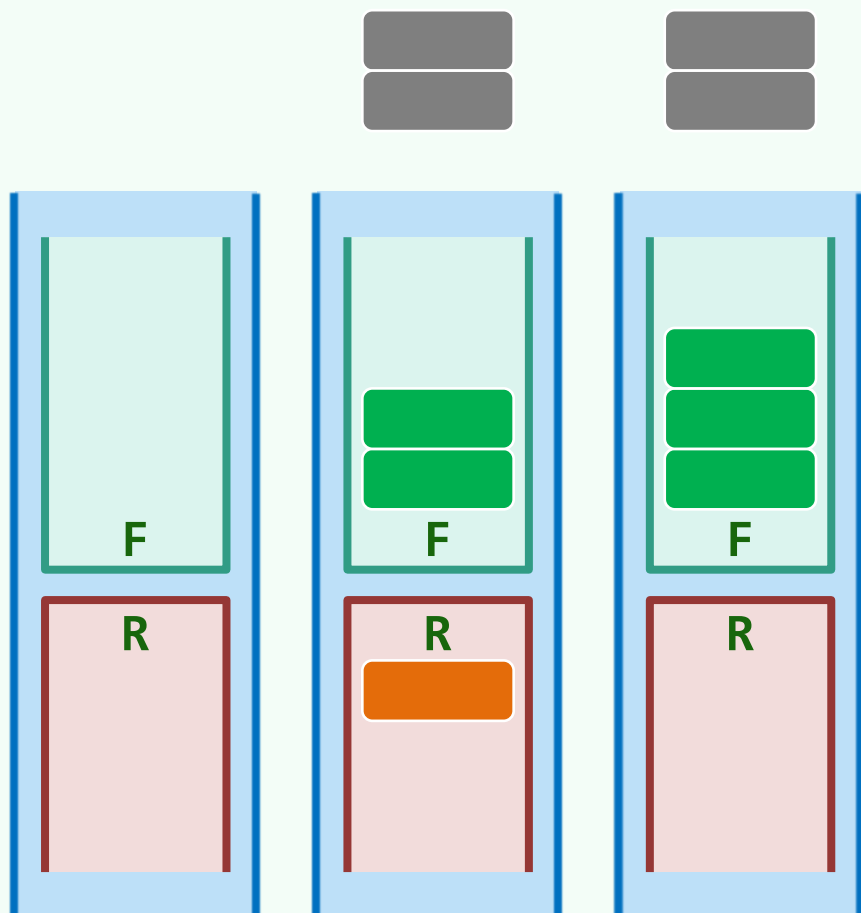
- **1** for its `enqueue()`
- **2** for transfer, and
- the last **1** for `dequeue()`

❖ Hence every operation is **pre-paid** and ...

❖ The structure will never run out of credit

❖ **Amortized cost** of any operation sequence involving **n** ITEMS is
 $4n = \mathcal{O}(n)$

Aggregate|聚合法



❖ Consider the moment when

d dequeue()'s and e enqueue()'s

have been done $// d \leq e$

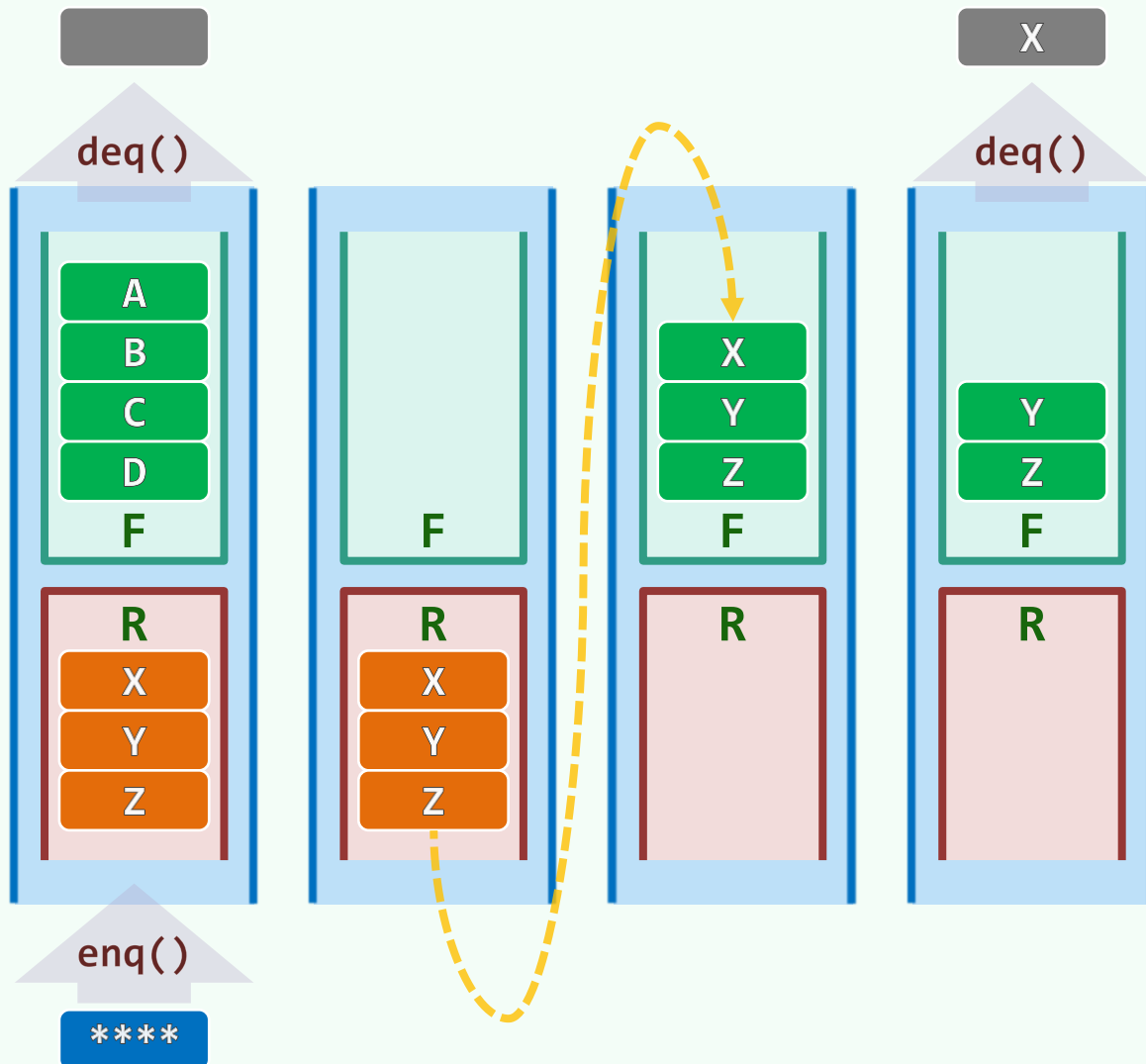
❖ The time cost for **ALL** the operations is

$$\leq 4 \cdot d + 3 \cdot (e - d) = 3e + d$$

❖ The amortized cost for each OPERATION is

$$\frac{3e + d}{e + d} < 3 = \mathcal{O}(1)$$

Potential | 势能法



❖ Consider the k^{th} operation

❖ Define $\Phi_k = |R_k| - |F_k|$

❖ Then $A_k = T_k + \Phi_k - \Phi_{k-1} \equiv 2$

❖ Hence

$$2n \equiv \sum_{k=1}^n A_k = \sum_{k=1}^n T_k + \Phi_n - \Phi_0$$

$$2n = T(n) + \Phi_n - \Phi_0 > T(n) - n$$

$$T(n) < 3n = \mathcal{O}(n)$$